

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

CodeQuest: A Game-Based Learning System for Software Engineering Education

by

B HARIS AHAMED **24BCE1136**

SHAILI H **24BCE1970**

A project report submitted
for the course of

BCSE301L — SOFTWARE ENGINEERING

in

B. Tech. COMPUTER SCIENCE AND ENGINEERING

October 2026

Table of Contents

1. Abstract....4
2. Introduction....5
2.1 Background and motivation....5
2.2 Problem statement....5
2.3 Objectives....5
2.4 Scope....6
2.5 Organisation of this report....6
3. Literature Survey....7
3.1 The effectiveness of gamified and game-based learning....7
3.2 Games in software engineering education....8
3.3 What the survey establishes....9
4. Existing Work and Systems....10
4.1 SimSE — the closest prior system....10
4.2 Programming-focused educational games....10
4.3 The gap this project occupies....11
5. Proposed Work and System....13
5.1 The central idea....13
5.2 The eight worlds....13
5.3 The five-stage world loop....14
5.4 The teaching phase....17
5.5 Progression and the Bug Lord....17
5.6 Design decisions deliberately taken....18
6. Software Requirements Specification....19
6.1 Overall description....19
6.2 Functional requirements....19
6.3 Non-functional requirements....21
6.4 Verification status....22
7. System Design and Architecture....23
7.1 Design goal....23
7.2 Layer model....23
7.3 The four autoloads....23
7.4 Class structure....24
7.5 The flow layer....25
7.6 Principal flows....25
7.7 The battle state machine....27
7.8 Data flow....29
7.9 Data model....30
7.10 Principal algorithms and their invariants....31
8. Technology Stack....33
8.1 Why the content formats differ....34
9. Working Modules....35

10. Description of Each Module....37
10.1 The data layer....37
10.2 The session layer....37
10.3 The battle layer....38
10.4 The presentation layer....39
11. Test Plan and Test Suite....41
11.1 Strategy....41
11.2 Risk analysis....41
11.3 Automated test suite....42
11.4 Manual test suite....43
11.5 Defect log....43
11.6 Exit criteria....45
11.7 What is deliberately not tested....45
12. Screenshots....46
12.1 Title and navigation....46
12.2 The teaching phase....50
12.3 The battle....53
12.4 Bugfall, the Bug Lord and victory....55
13. Conclusion....58
14. Future Work....60
14.1 Evaluation....60
14.2 Known defects and limitations....60
14.3 Content and features....60
14.4 Research directions....61
15. References....62
15.1 Software and tools....63
15.2 Project documents....63
16. Appendix: Implementation....64
16.1 Project structure....64
16.2 The flow layer — world_flow.gd....64
16.3 Progression — the idempotency guard....65
16.4 The battle state gate....65
16.5 Automatic audio wiring....65
16.6 A lesson beat....66
16.7 Running the automated suite....66

1. Abstract

Software engineering is taught as a sequence of process phases — requirements, design, implementation, testing, configuration management, maintenance and deployment — yet the subject is commonly assessed through written examinations that reward recall of definitions rather than the judgement those phases demand. Students can state what a Software Requirements Specification is without ever having felt the cost of an ambiguous requirement.

This report presents **CodeQuest**, a two-dimensional role-playing game built in Godot 4.7.2 that restructures that syllabus as a playable campaign. Eight worlds map one-to-one onto the phases of the software development life cycle. Each world is guarded by a themed adversary — the Ambiguous Client of Reqoria, the Infinite Loop Beast of Codexis, the Merge Conflict Titan of Gitforge — and each is preceded by a taught lesson delivered by a mentor character, Sifu, The Wise. The central design decision is that **assessment is the combat mechanic itself**: a correct answer to a multiple-choice question is the hero's attack, and an incorrect answer is the enemy's. Progress is gated rather than scored. Defeating each of the seven Generals removes 1000 points from the persistent 8000-point health pool of the final adversary, the Bug Lord, whose encounter is a timed pass-or-fail examination rather than a conventional fight.

The system is built on a three-layer architecture — data, session and presentation — in which all content is defined as Godot `Resource` objects, a question bank in CSV and lesson scripts in JSON, so that a new world can be added without modifying gameplay code. Forty-four architecture decision records document the reasoning behind each significant choice. Verification combines five automated headless checks comprising 1159 assertions with a twenty-eight case manual suite; the resulting defect log of fourteen entries is analysed in this report as evidence for the testing strategy adopted.

A survey of the literature establishes that gamified instruction produces real but modest learning gains, and that existing educational games overwhelmingly teach programming rather than software engineering process. The one substantial exception, SimSE, is a project-management simulation whose own evaluation reported high drop-off and repetitive play. CodeQuest's contribution is therefore narrower and more precise than a claim of novelty: it applies **content gamification** — in which the assessment instrument is the game mechanic — within a narrative role-playing structure, to software engineering process areas that recent systematic reviews identify as under-served.

2. Introduction

2.1 Background and motivation

The software development life cycle is the organising framework of almost every introductory software engineering curriculum. It is also, for most students, the least tangible part of it. Writing code produces immediate and unambiguous feedback: the program compiles or it does not, the test passes or it fails. Process knowledge offers no such loop. A student can produce a plausible requirements document and receive a mark weeks later, with no intervening moment at which a poor requirement visibly costs anything.

The consequence is familiar to anyone who has marked such a course. Terminology is reproduced accurately and applied poorly. “The system should be user-friendly” is written as a requirement by students who can, if asked directly, define verifiability correctly. The knowledge exists; the judgement does not, because nothing in the learning process ever exercised it.

Game-based learning offers one response. A game can compress a feedback loop that is naturally weeks long into seconds, and it can attach a visible consequence to a judgement rather than to a definition. The evidence for its effectiveness is real but, as Section 3 sets out in detail, more modest and more qualified than its advocates usually report.

2.2 Problem statement

Existing educational games in computing almost universally teach **programming**. CodeCombat, CodinGame, Lightbot and Human Resource Machine all take the same essential shape: the player writes or sequences instructions, and the game executes them. None of them models requirements elicitation, architectural trade-off, version control conflict, deployment risk or the maintenance of unfamiliar code — the activities that occupy most of a professional engineer’s working life and most of an SE syllabus.

Where software engineering process *has* been gamified, systematic reviews find the treatment is almost always **structural**: points, badges and leaderboards added around conventional material. The material itself is unchanged. The problem this project addresses is therefore not the absence of games in software engineering education, but the absence of games in which the educational content and the game mechanic are the same object.

2.3 Objectives

1. Design a complete, playable role-playing campaign whose structure maps one-to-one onto the eight phases of the software development life cycle.
2. Make assessment the core mechanic rather than an interruption of it, so that answering a question correctly is the player’s means of progress.

3. Deliver explicit instruction before assessment, so that the game teaches rather than merely tests.
4. Build the system on a data-driven architecture in which worlds, enemies, questions and lessons are content files, not code.
5. Gate progression so that a player cannot reach the final encounter without having demonstrated competence in every preceding phase.
6. Document every significant design decision, and verify the system against a written requirements baseline.

2.4 Scope

In scope. Eight playable worlds with distinct art, questions and lessons; a mentor-delivered teaching phase for every world; a turn-based battle system driven by multiple-choice assessment; a timed final examination against the Bug Lord; persistent progression saved between sessions; a reference library of all eight lessons; audio and visual feedback; and a documented architecture with a traceable requirements baseline.

Out of scope. Multiplayer or networked play; an instructor-facing analytics dashboard; adaptive difficulty; a content-authoring user interface (content is authored by editing CSV, JSON and Godot resource files directly); mobile or web distribution; and a controlled classroom study of learning outcomes. The last of these is the most significant omission and is discussed in Section 16.

2.5 Organisation of this report

Section 3 surveys the research literature on game-based learning and on software engineering education specifically. Section 4 examines existing systems and locates the gap this project occupies. Section 5 describes the proposed system and Section 6 states its requirements. Sections 7 to 10 cover the architecture, the technology stack and the module decomposition. Section 11 presents the test plan and the defect evidence behind it, and Section 12 the resulting system in screenshots. Sections 13 and 14 conclude and identify future work.

3. Literature Survey

This survey addresses three questions in turn. Does game-based learning work, and how well? Has it been applied to software engineering education? And what do the existing systems actually teach? The answers determine what this project can honestly claim.

3.1 The effectiveness of gamified and game-based learning

Sailer and Homner (2020) conducted a random-effects meta-analysis of gamification in learning. They report a significant cognitive effect of g = .49 across nineteen comparisons, a motivational effect of g = .36 across sixteen, and a behavioural effect of g = .25 across nine. These are the figures most often quoted in support of educational games.

The qualification matters more than the headline. When the authors restricted their analysis to studies of higher methodological rigour, the cognitive effect held at g = .42, but **the motivational and behavioural effects became statistically non-significant**. Heterogeneity across studies was substantial (I^2 between 64% and 75%), delayed post-tests were rare, and the authors note the possibility of publication bias. The claim that gamification improves *motivation* — precisely the claim a project such as this one is tempted to make — is the one their data supports least.

Gui et al. (2023) reach a more positive conclusion within STEM specifically. Across 136 effect sizes from 86 studies they find digital game-based STEM learning outperforms conventional instruction at g = 0.624. A second analysis, of 81 effect sizes from 44 studies, examines what happens when game-design elements are *added* to an existing game, finding g = 0.301. Their moderator analysis is the finding most relevant to this project: **game elements that support content learning outperformed elements added for the sake of gaming experience**. This directly informs CodeQuest's central design decision, described in Section 5.

Almeida et al. (2023) provide the necessary counterweight. Their systematic mapping of 87 papers catalogues the *undesired* effects of gamification in educational software. Badges, leaderboards, competition and points are the elements most frequently implicated. The harms most often reported are no effect at all, worsened performance, damage to motivation, lack of understanding, and perceived irrelevance; gaming-the-system and outright cheating recur as ethical concerns. A focus group with practitioners building a gamified educational product found the developers largely unaware of these risks. Any design that reaches for points and badges first should be read against this paper.

Ishaq et al. (2021) review 67 studies of gamified mobile language-learning applications, and are useful here chiefly for their explicit statement of methodological limits: a title-only search, a 2010–2020 window, English-language sources only, and several included studies scoring poorly on the authors' own quality assessment.

3.2 Games in software engineering education

Souza et al. (2018) conducted a systematic mapping study of game-related methods in software engineering education covering 156 primary studies published between 1974 and 2016. They classify the field into game-based learning (approximately 88 studies), game-development-based learning (approximately 60) and gamification (approximately 11).

Their distribution by knowledge area is the single most important finding for this project, and it cuts against the obvious claim of novelty. **Software Process is the most heavily studied area, appearing in 92 of the 156 studies — 59%.** Software Design follows at 38.5% and Professional Practice at 30.8%, while Software Quality and Software Modelling appear in only 6.4% each. It is therefore false to assert that software engineering process has been neglected by educational game research. The authors do, however, report that evaluation evidence across the field is weak and that there is no standardised set of learning goals or game classification.

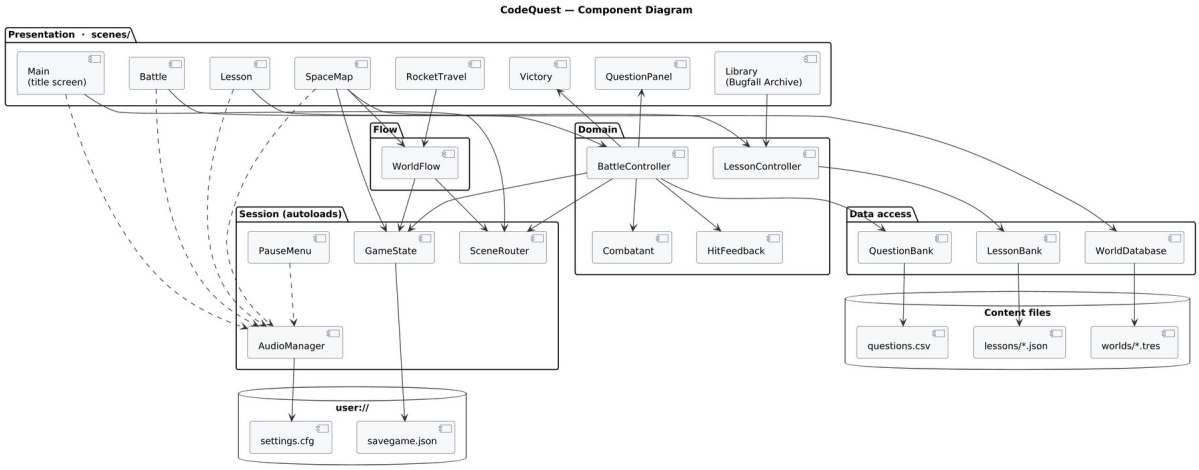


Figure 1: The architectural shape this survey ultimately informed: content and mechanic in the same layer.

Kosa et al. (2016) screened 350 papers from a fifteen-year window, retaining 53 primary studies, and propose a five-part taxonomy: games students play, games students build as coursework, curriculum proposals, new approaches and tools, and other. CodeQuest sits unambiguously in the first category.

Tonhão et al. (2023) carried out a tertiary study — a review of ten secondary studies — on gamification in software engineering education, and it is this paper that defines the space CodeQuest actually occupies. Three of its findings are directly relevant. First, nearly all existing work employs **structural** gamification, which alters the environment around the material while leaving the material itself untouched; **no study included in their review used content gamification alone**. Second, Software Testing and Software Quality are the most gamified areas, while Software Maintenance and Software Configuration Management appear in very few studies. Third, narrative, storytelling and

time pressure are rarely explored as mechanisms. The authors also report harms arising from poorly planned gamification, echoing Almeida et al.

Wang et al. (2025) describe DevCoach, a web environment in which students progress through five SDLC phases accompanied by six large-language-model agents playing development roles. In a between-subjects study with twenty participants, the DevCoach group improved by 2.2 points pre-to-post against 0.2 for a baseline using conventional online resources (Mann-Whitney $U = 22.0$, $p = 0.034$). The study is small — one institution, one project, a single session — but it is the closest contemporary work to CodeQuest on the specific idea of phase-structured SDLC learning, differing in that it is an agent-based tutor rather than a game.

3.3 What the survey establishes

- Gamified instruction produces **real but modest** cognitive gains. The motivational benefit, which is the most commonly asserted, is the least robust to methodological scrutiny (Sailer and Homner, 2020).
- Game elements tied to the **content** outperform elements added for the feel of a game (Gui et al., 2023).
- Badge-and-leaderboard gamification carries **documented harms**, and practitioners are frequently unaware of them (Almeida et al., 2023).
- Software process is **not** an untouched area in SE education games; it is the most studied one (Souza et al., 2018).
- However, content gamification, narrative, and the Maintenance and Configuration Management areas remain **genuinely under-explored** (Tonhão et al., 2023).

The design of CodeQuest follows from the second and fifth of these points. The assessment instrument is made the game mechanic rather than a reward wrapped around one, and two of the eight worlds are given to Maintenance and Configuration Management. Section 4 examines the specific systems against which the design must be compared.

4. Existing Work and Systems

Five existing systems bear on this project. Four are commercial or widely used educational games that teach programming; the fifth, SimSE, is an academic system that teaches software engineering process and is therefore the direct point of comparison.

4.1 SimSE — the closest prior system

SimSE, developed by Navarro and van der Hoek at the University of California, Irvine and first presented in 2004, is an interactive simulation game for software engineering education. The player takes the role of a **project manager** in a graphical virtual office: hiring and firing staff, assigning tasks, purchasing tools and monitoring progress. It explicitly targets process lessons that are difficult to convey in lecture — Brooks' Law, the benefits of configuration management, the selection of an appropriate life-cycle model, and the trade-offs between cost, schedule and quality. It ships with a model builder so that instructors can author their own process models.

SimSE demonstrates that the obvious claim — that educational games teach coding rather than software engineering — is false. Any survey that omitted it would be incomplete, and this report states it plainly for that reason.

What makes SimSE useful to this project is its own published evaluation (CSEET 2007), which is candid and mixed. Nineteen students were randomised across three conditions: playing SimSE, reading equivalent material, and attending a lecture. Time spent playing correlated strongly with gains in process knowledge ($r = 0.81$, $p < 0.001$). However:

- **No SimSE participant completed the assigned play**, and the SimSE group improved least from pre-test to post-test — the reading and lecture groups improved more.
- Eleven of the thirty initially recruited subjects dropped out of the study.
- Inadequate in-game instruction caused frustration, and extended play was reported as repetitive.
- Students with no prior software engineering background learned the least from it.

The authors conclude that SimSE functions as a complement to instruction rather than a replacement for it. The pattern their data describes — strong effect *when played*, combined with players not playing it — is a retention problem, and it is precisely the problem that the narrative and progression structure of a role-playing game is designed to address.

4.2 Programming-focused educational games

The remaining systems are included because they are the ones students and instructors actually encounter, and because the contrast with them is instructive.

System	Teaches	Mechanism	SE process?
CodeCombat	Python, JavaScript, HTML	Typed code drives a character through puzzle levels in the browser	No
CodinGame	25+ languages, algorithms	Puzzles, bot programming, competitive multiplayer events	No
Lightbot	Computational thinking — procedures, loops, conditionals	Sequencing instructions to light tiles	No
Human Resource Machine	Assembly-language concepts	Office metaphor: inbox and outbox, registers, conditional jumps	No
Duolingo	Natural languages (cited here for its motivational model only)	Streaks, XP, leagues, hearts, gems, friend quests	N/A

Table 1: Existing educational systems and what each actually teaches

CodeCombat has been evaluated academically. Kroustalli and Xinogalos (2021) compared it against lectures plus a conventional Python environment with 59 lower-secondary students. The CodeCombat group scored higher, but **the difference was not statistically significant**, and while the Technology Acceptance Model ratings for ease of use and usefulness were positive, students' behavioural intention to continue using the system was neutral. This is a useful corrective to the assumption that a well-made educational game automatically outperforms direct instruction.

Duolingo is included not as an educational comparator but as the clearest working example of a retention loop. Its documented mechanics — daily streaks with purchasable protection, experience points with time-of-day and milestone bonuses, weekly promotion leagues, hearts lost on incorrect answers, and an in-game currency — constitute the reference design for sustaining voluntary engagement over months. CodeQuest borrows the principle of gated, visible progression from it while deliberately omitting the competitive elements that Almeida et al. associate with harm.

4.3 The gap this project occupies

Sections 3 and 4 locate CodeQuest against the existing systems as the intersection of four design conditions. No one of the four is novel in isolation; no reviewed system satisfies all four simultaneously:

1. **Content gamification rather than structural.** The assessment instrument is the game mechanic itself, not a reward wrapped around unchanged material. Tonhão et al. found no reviewed study using content gamification alone.
2. **Narrative role-playing structure.** The same review reports narrative and storytelling as rarely explored. SimSE is a management simulation; the programming games are puzzle sequences.
3. **Explicit instruction preceding assessment.** SimSE's evaluation attributes part of its difficulty to inadequate in-game instruction; CodeQuest delivers a full taught lesson before every encounter.
4. **Coverage of under-served process areas.** Maintenance and Configuration Management are near-absent from the reviewed literature. CodeQuest devotes a full world to each — Legacyon and Gitforge respectively.

These four conditions are the design inputs to Section 5, and each is traceable to a specific mechanic: content gamification to the battle loop, narrative to the world and General structure, instruction-before-assessment to the five-stage world loop, and process coverage to the world to SDLC-phase mapping.

5. Proposed Work and System

5.1 The central idea

CodeQuest restructures an eight-phase software engineering syllabus as a role-playing campaign across eight worlds. The single decision from which the rest of the design follows is this: **the assessment instrument is the combat mechanic**. The player has no attack button. A multiple-choice question appears, and the answer selected determines who is struck. A correct answer is the hero's attack and removes health from the General; an incorrect answer is the General's attack and removes health from the hero.

This is the distinction Section 3 identified as under-explored. In structural gamification the quiz is interrupted by a reward; here there is no separable quiz and no separable reward. Answering *is* playing. It also follows Gui et al.'s finding that elements supporting content learning outperform elements added for gaming experience, because in this design there are no elements of the second kind to add.

5.2 The eight worlds

Each world corresponds to one phase of the software development life cycle and is guarded by an adversary whose name personifies the characteristic failure of that phase.

#	World	SDLC phase	General	Material taught and examined
1	Reqoria	Requirement Analysis	Ambiguous Client	Functional vs non-functional requirements, verifiability, elicitation, the SRS as a contract, scope creep
2	Architara	System Design / Architecture	Blueprint Wraith	Coupling and cohesion, layering, design patterns, architectural trade-off, design documentation
3	Codexis	Implementation	Infinite Loop Beast	Coding standards, readability, code review, defensive programming, technical debt at the statement level
4	Testara	Testing	Null Pointer Phantom	Unit, integration and system testing, black- and white-box technique, test cases, coverage, regression
5	Gitforge	Version Control / Configuration Management	Merge Conflict Titan	Branching models, commits, merge and rebase, conflict resolution, release tagging, baselines
6	Legacyon	Maintenance	Technical Debt King	Corrective, adaptive, perfective and preventive maintenance, refactoring, reading unfamiliar code
7	Deployon	Deployment / DevOps	Production Nightmare	Build pipelines, environments, continuous integration, release strategy, rollback, monitoring

#	World	SDLC phase	General	Material taught and examined
8	Bugfall	Debugging / Final Examination	The Bug Lord	Defect classification, root-cause analysis, reproduction, and a cumulative examination of all seven phases

Table 2: The eight worlds of CodeQuest and their mapping onto the SDLC

Bugfall is deliberately different. It has **no General**. Its planet holds the Learning Archive — a shelf from which any of the eight lessons may be re-read — and the route to the Bug Lord.

5.3 The five-stage world loop

Every world is played through the same five stages, which give the project its pedagogical shape:

1. **LEARN.** Sifu, The Wise delivers a lesson of ten or eleven beats — dialogue, concepts, side-by-side comparisons, worked examples and numbered procedures.
2. **UNDERSTAND.** A comprehension check inside the lesson, answered before the fight, with the reasoning revealed rather than a bare mark.
3. **APPLY.** The battle begins. Questions are drawn from that world’s pool and answered under the combat mechanic.
4. **TEST.** The battle is itself the test: the player must reduce the General to zero health before the General does the same to them.
5. **PROGRESS.** Victory clears the world, unlocks the next, and removes 1000 points from the Bug Lord’s persistent health.

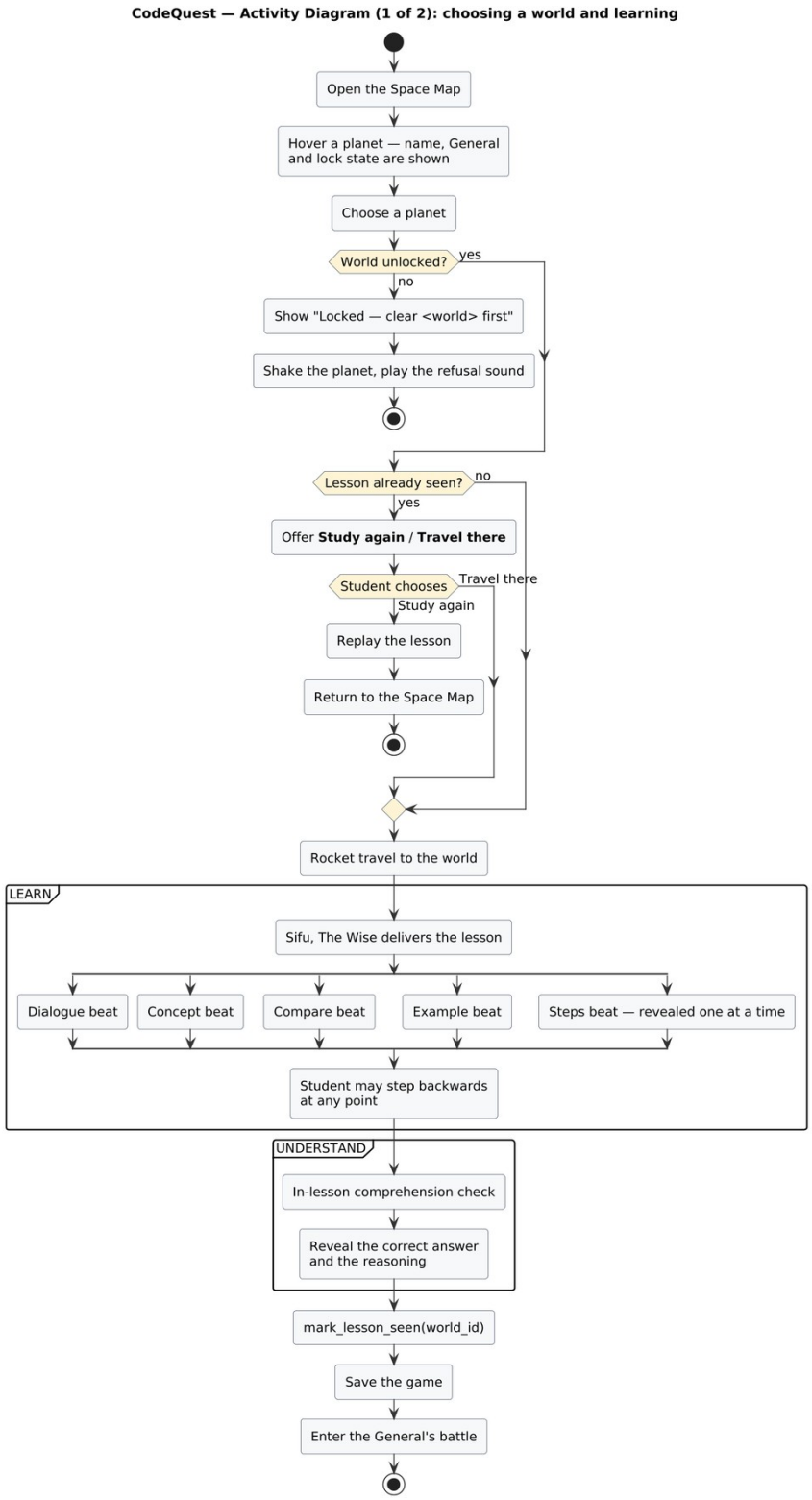


Figure 2: Activity diagram (1 of 2): world selection, travel and the teaching phase

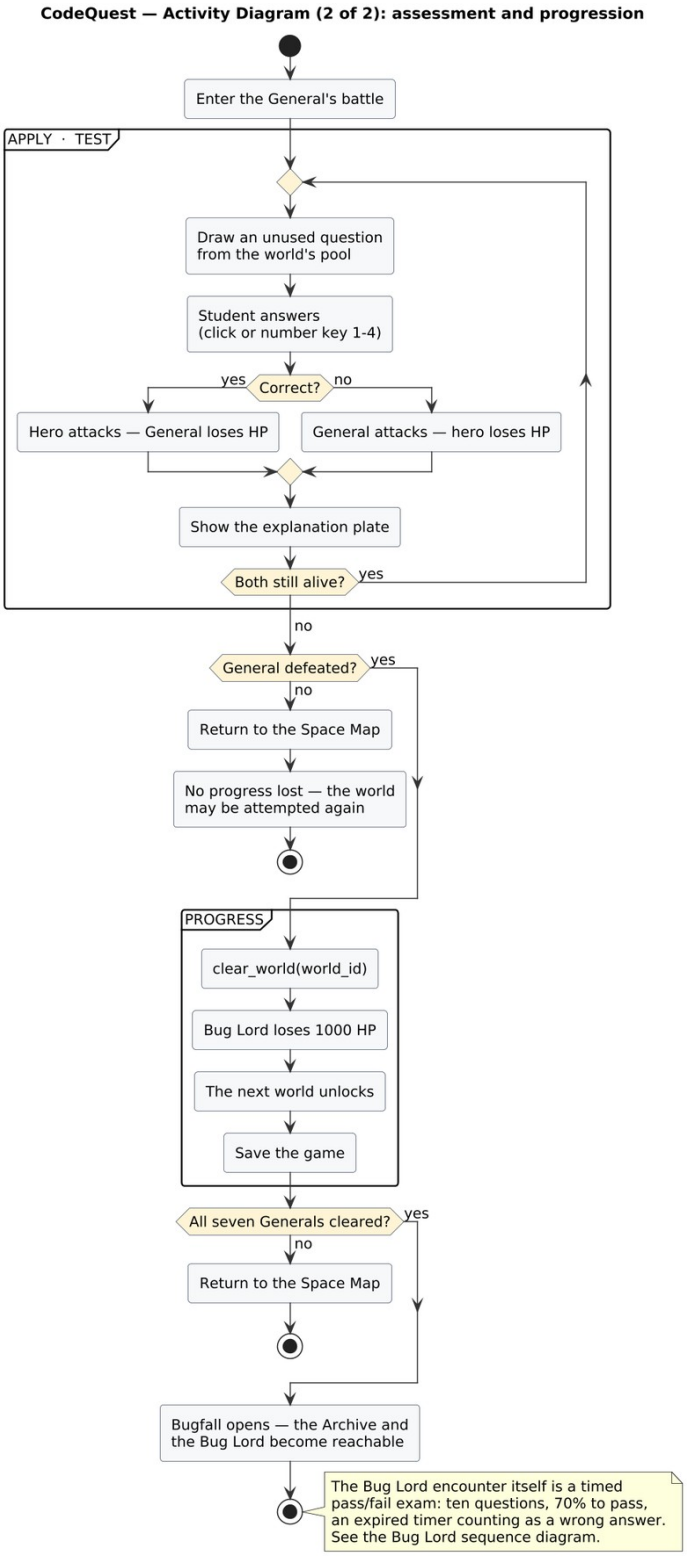


Figure 3: Activity diagram (2 of 2): assessment and progression

5.4 The teaching phase

A game that only tests is a quiz with decoration. CodeQuest therefore teaches first. The lesson player supports seven distinct beat types, each rendered differently:

Beat type	Purpose
`dialogue`	Sifu speaks in character; sets the stakes for the world
`concept`	A titled definition with supporting prose and key terms
`compare`	Two columns side by side — for instance functional against non-functional requirements
`example`	A worked contrast: an ambiguous requirement beside the same requirement pinned down
`steps`	A numbered procedure, revealed one step at a time rather than all at once
`check`	A comprehension question with the reasoning given after the answer
`encounter`	The transition into the battle

Table 3: The seven lesson beat types

Lessons may be navigated backwards as well as forwards, and a lesson already studied may be re-read from its own planet or from the Archive. The reason is pedagogical rather than convenience: a student who has forgotten Reqoria’s terminology by the time they reach world five should not have to finish the game to revisit it.

5.5 Progression and the Bug Lord

Progression is **gated, not scored**. There is no points total and no leaderboard — a deliberate omission given the harms Almeida et al. associate with those elements. World $n + 1$ unlocks when world n is cleared, and a locked world states which world unlocks it rather than failing silently.

The Bug Lord begins the campaign with 8000 health. Each General defeated removes 1000, so a player who has cleared all seven arrives at Bugfall facing an adversary reduced to 1000. This makes the whole campaign legible as a single bar on the Space Map: the player can see the final enemy weakening as they learn.

The final encounter is **a timed examination, not a health contest**. Ten questions are drawn from across all eight worlds, each under a countdown; an expired timer counts as a wrong answer. Passing requires 70%. A fourth incorrect answer both fails the examination and empties a full-health hero, so the two failure conditions coincide by design rather than by accident.

5.6 Design decisions deliberately taken

- **No score, no leaderboard, no badges.** Progression is the only reward. See Section 3.1 on documented harms.
- **Health resets between encounters.** A world failed costs time, not the campaign; the player is never soft-locked by accumulated damage.
- **Clearing a world is idempotent.** Replaying a cleared world cannot damage the Bug Lord twice, preventing the campaign being finished from world one.
- **No question repeats within an encounter.** Questions are drawn from a pool and moved to a used list.
- **Every wrong answer is explained.** The explanation plate names the correct option and why it is correct, so a failed question still teaches.

6. Software Requirements Specification

This section states the requirements baseline against which the system was built and verified. The full traceability matrix, which records the implementing component and verification status of every item, is maintained in `docs/REQUIREMENTS_TRACE.md` in the project repository and currently contains **137 numbered requirements**.

6.1 Overall description

Product perspective. CodeQuest is a self-contained single-player desktop application. It depends on no server, no network connection and no external service. All content ships with the build; all state persists to the user's local application data directory.

User classes. Two. The *student* plays the campaign and is the primary user. The *content author*, typically an instructor, adds or edits worlds, questions and lessons by editing files; no authoring user interface is provided and none is required, because the content formats were chosen to be editable by hand.

Operating environment. Windows, macOS or Linux desktop; OpenGL 3.3 compatibility renderer; keyboard and mouse. Design resolution 1152 × 648 with `canvas_items` stretch, so the window may be resized freely.

Assumptions and dependencies. The player is a student of an introductory software engineering course; the content assumes no prior industry experience. The game assumes a single player per save file.

6.2 Functional requirements

Requirements are grouped by the Master Brief section they derive from. The identifiers are those used in the traceability matrix.

ID	Requirement
FR-1	**Game concept.** The system shall present an eight-world campaign in which each world corresponds to one phase of the software development life cycle.
FR-1.1	The system shall deliver instruction before assessment in every world.
FR-1.2	The system shall use multiple-choice assessment as the sole means of attack in battle.
FR-2	**Worlds and Generals.** The system shall provide eight worlds, seven of which are guarded by a uniquely themed General.
FR-2.1	Each world shall declare an identifier, display name, SDLC phase, map position and order index.
FR-2.2	World 8 (Bugfall) shall have no General and shall instead give access to the Archive and the Bug Lord.
FR-3	**Space Map.** The system shall present all eight worlds on a navigable star

ID	Requirement
	map.
FR-3.1	Hovering a planet shall display its name, its General and, if locked, the world that unlocks it.
FR-3.2	Clicking a locked world shall refuse the action visibly and audibly rather than silently.
FR-3.3	The map shall display campaign progress as the Bug Lord’s remaining health and the count of worlds cleared.
FR-3.4	A world whose lesson has been studied shall offer a choice of studying again or travelling.
FR-4	**Battle system.** The system shall run a turn-based encounter between the hero and the world’s General.
FR-4.1	A correct answer shall cause the hero to attack for the hero’s attack damage.
FR-4.2	An incorrect answer shall cause the enemy to attack for the enemy’s attack damage.
FR-4.3	Each question shall be drawn without repetition from that world’s pool.
FR-4.4	The system shall display an explanation after every answer, correct or not.
FR-4.5	Answers shall be selectable by mouse or by number keys 1 to 4.
FR-4.6	The encounter shall end when either combatant reaches zero health.
FR-5	**Teaching phase.** The system shall deliver a lesson of ten or eleven beats before each first encounter.
FR-5.1	The lesson player shall support dialogue, concept, compare, example, steps, check and encounter beats.
FR-5.2	The player shall be able to move backwards through a lesson.
FR-5.3	A studied lesson shall be re-readable from its own planet and from the Bugfall Archive.
FR-5.4	The Archive shall indicate which lessons have been studied and how many of the eight.
FR-6	**Bug Lord system.** The Bug Lord shall hold persistent health across the campaign, beginning at 8000.
FR-6.1	Clearing a world shall reduce the Bug Lord’s health by exactly 1000.
FR-6.2	Clearing an already-cleared world shall have no effect on the Bug Lord’s health.
FR-6.3	The Bug Lord encounter shall be reachable only when all seven Generals have been defeated.
FR-7	**Final examination.** The Bug Lord encounter shall be a timed pass-or-fail examination of ten questions.

ID	Requirement
FR-7.1	Each question shall carry a countdown; expiry shall be treated as an incorrect answer.
FR-7.2	The pass threshold shall be 70% of the ten questions.
FR-8	**Persistence.** The system shall save progress automatically and restore it on next launch.
FR-8.1	The save shall record defeated generals, lessons seen, and the Bug Lord’s health.
FR-8.2	The save shall carry a version field and shall refuse a file of a different version rather than apply it partially.
FR-9	**Audio and settings.** The system shall provide sound effects and background music, with a mute setting that persists between sessions.
FR-10	**Pause.** The player shall be able to pause at any point during play, by key and by an on-screen control.

Table 4: Principal functional requirements (abridged from the 137-item traceability matrix)

6.3 Non-functional requirements

ID	Category	Requirement
NFR-1	Usability	Every interactive control shall give visible feedback on hover and on activation. A refused action shall state why.
NFR-2	Usability	Text shall remain legible at the design resolution without the player needing to resize the window.
NFR-3	Performance	The system shall maintain interactive frame rates on integrated graphics using the OpenGL compatibility renderer.
NFR-4	Reliability	A corrupt or foreign save file shall be refused outright rather than applied in part.
NFR-5	Reliability	The battle state machine shall not accept an answer outside the question state, so that a repeated input cannot resolve twice.
NFR-6	Maintainability	No single script shall carry unrelated responsibilities; content shall be addable without modifying gameplay code.
NFR-7	Maintainability	Every significant design decision shall be recorded in an architecture decision record.
NFR-8	Portability	The project shall build for desktop platforms from a single codebase with no platform-specific code paths.
NFR-9	Testability	Data integrity and progression arithmetic shall be verifiable without a rendering context.

ID	Category	Requirement
NFR-10	Visual polish	Combat shall convey weight through anticipation, follow-through, impact feedback and screen shake.
NFR-11	Audio	Sound shall be present at every significant interaction and state change, and shall be mutable.
NFR-12	Documentation	The project shall produce the standard software engineering documents and diagrams for the system as built.

Table 5: Non-functional requirements

6.4 Verification status

As of the build described in this report, the traceability matrix records:

Status	Meaning	Count
Met	Implemented and verified	114
Partial	Implemented in part; the remainder is stated in the row	6
Diverged	Deliberately built differently; the deciding ADR is named	6
Open	Not implemented, still intended	9
Deferred	Explicitly out of scope for this version	2
Total		**137**

Table 6: Requirements verification status

Of the nine items still open, eight were the software engineering diagrams required by the brief, which this report delivers; the ninth is camera zoom during combat, discussed in Section 14.

7. System Design and Architecture

7.1 Design goal

The architecture serves one objective above the rest: **adding a world must not require changing gameplay code.** A new world should be a resource file, a block of rows in the question bank and a JSON lesson. Everything else in the design follows from holding that line.

7.2 Layer model

The system is organised into three layers with strictly one-way dependencies. Presentation may read data; data knows nothing of presentation.

Layer	Contents	May depend on
Data	`WorldData`, `WorldDatabase`, `EnemyData`, `HeroData`, `QuestionData`, `QuestionBank`, `LessonBank` — plus the content files they load	Nothing
Session	The four autoloads: `GameState`, `SceneRouter`, `AudioManager`, `PauseMenu`	Data
Presentation	The eight scenes and their controllers, including `BattleController`, `LessonController`, `SpaceMap`	Session, Data

Table 7: The three-layer model and its dependency rule

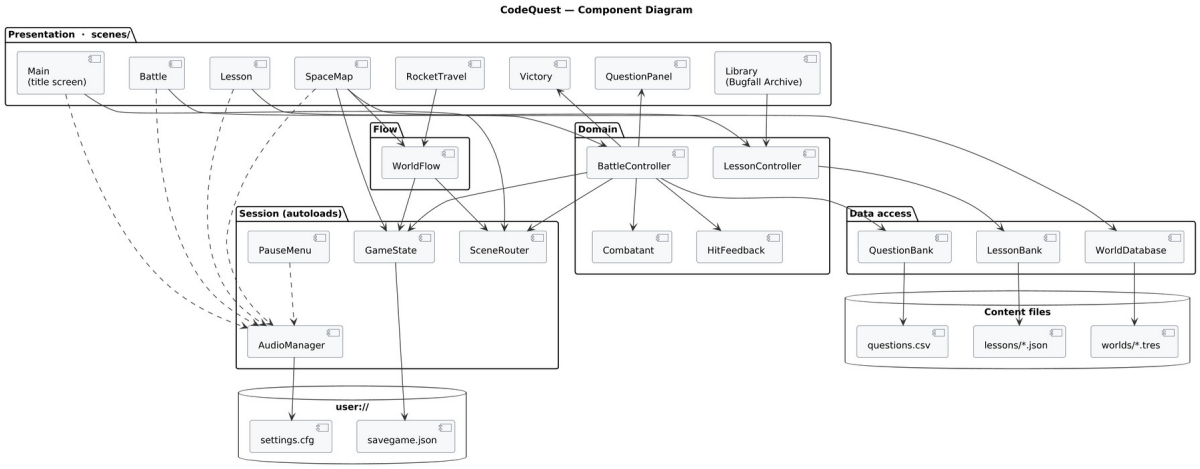


Figure 4: Component diagram: presentation, session, flow, domain and data access

7.3 The four autoloads

Godot autoloads are singletons instantiated before any scene and persisting across scene changes. Four were justified; nothing else was made global.

Autoload	Responsibility	Why it must be global
`GameState`	Campaign progression — which worlds are cleared, which lessons seen, the Bug Lord's health; save and load	Progress outlives every scene
`SceneRouter`	All scene changes, with a transition and an optional setup callback applied before the new scene enters the tree	A scene cannot own the transition out of itself
`AudioManager`	A ten-voice sound effect pool, two music players with crossfade, persisted settings; hooks every Button in the tree on creation	Music must survive the scene change it plays under
`PauseMenu`	Escape key and on-screen pause control, over a dimmed scene, on its own canvas layer	Must be reachable from any scene without each scene implementing it

Table 8: The four autoloads and the justification for each

`SceneRouter` deserves particular note. Its `goto_scene` accepts a path, an optional `Callable` that configures the incoming scene, a transition style and a tint. The callable is invoked after the scene is instantiated but **before it is added to the tree**, so a scene has its configuration in place before `_ready()` runs. This removes the usual alternative — a global "pending request" variable that the next scene reads and must remember to clear.

```
# SceneRouter.goto_scene — the payload-passing contract
var configure := func(scene: Node) -> void:
    scene.setup(world_data)

SceneRouter.goto_scene("res://scenes/travel/RocketTravel.tscn", configure)
```

7.4 Class structure

The principal types and their relationships are shown below. Resource subclasses carry only data and exported fields; behaviour lives in the controllers.

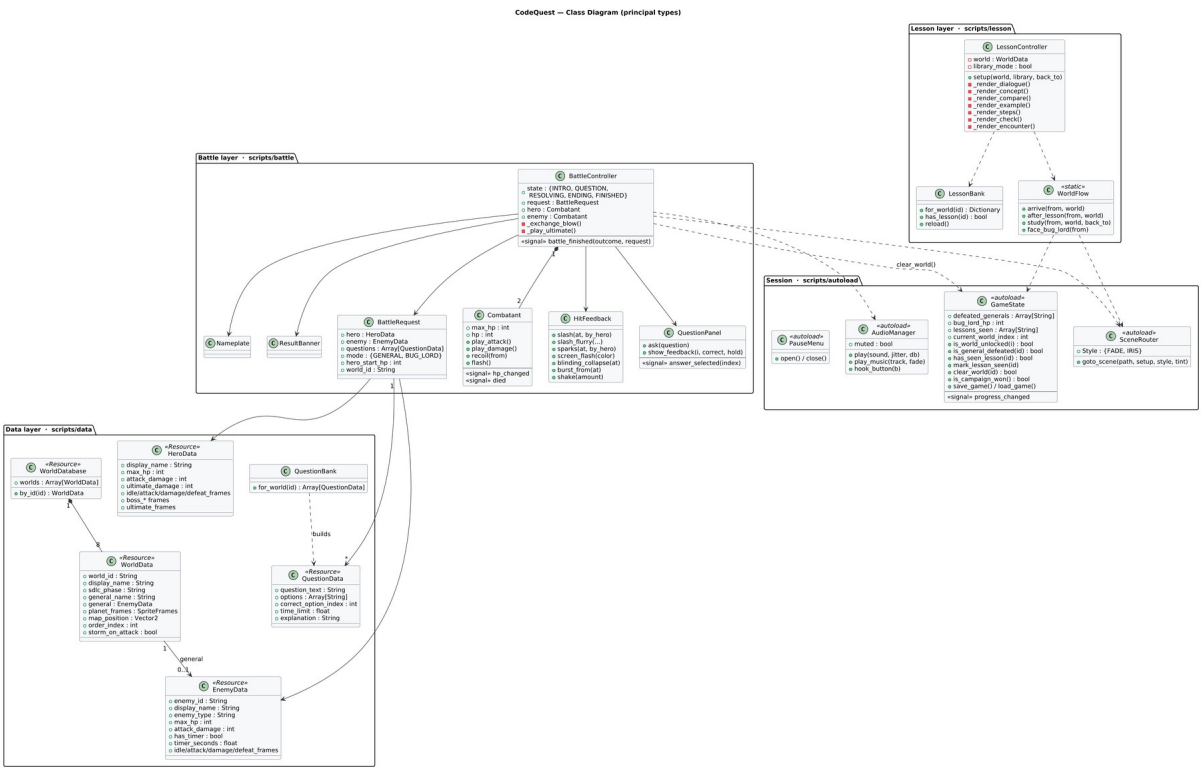


Figure 5: Class diagram of the principal types

7.5 The flow layer

A single static class, `WorldFlow`, decides what follows what. Without it, the knowledge that a lesson precedes a battle would be duplicated across the Space Map, the rocket travel scene and the lesson controller, and the rule could drift between them.

```
static func arrive(from: Node, world: WorldData) -> void
static func after_lesson(from: Node, world: WorldData) -> void
static func study(from: Node, world: WorldData, back_to: String = "") -> void
static func face_bug_lord(from: Node) -> void
```

`arrive()` is the one that matters: it checks whether a lesson exists for the world and whether it has been seen, and routes to the lesson or straight to the battle accordingly. Every caller that needs to enter a world calls `arrive()` and nothing else.

7.6 Principal flows

The two encounter types share the same controller and differ by a mode flag on the battle request. The sequence of a General fight is shown first.

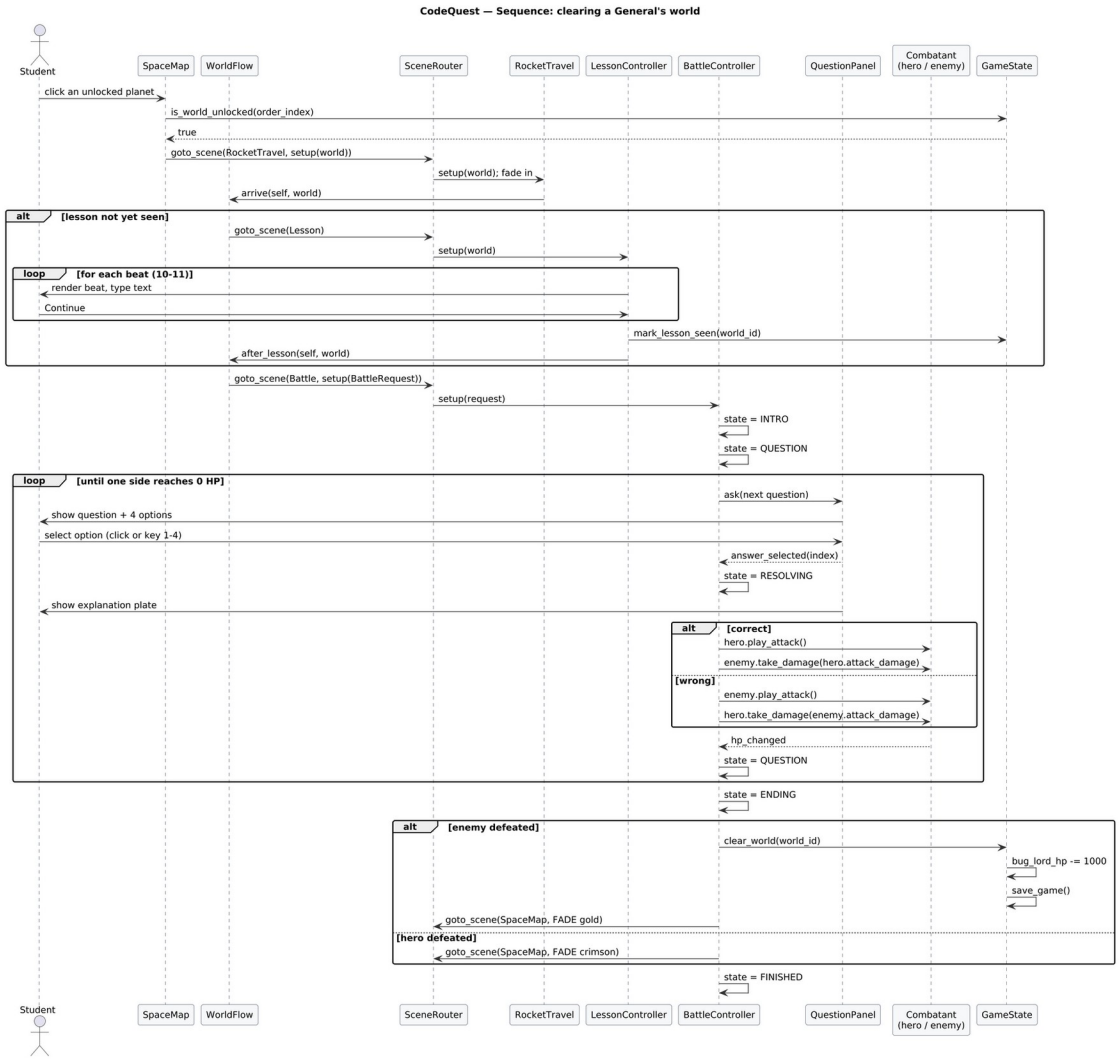


Figure 6: Sequence diagram: clearing a General's world from map click to save

The Bug Lord encounter reuses the same loop with three differences: a per-question timer, a fixed count of ten questions rather than a health race, and a pass threshold evaluated at the end.

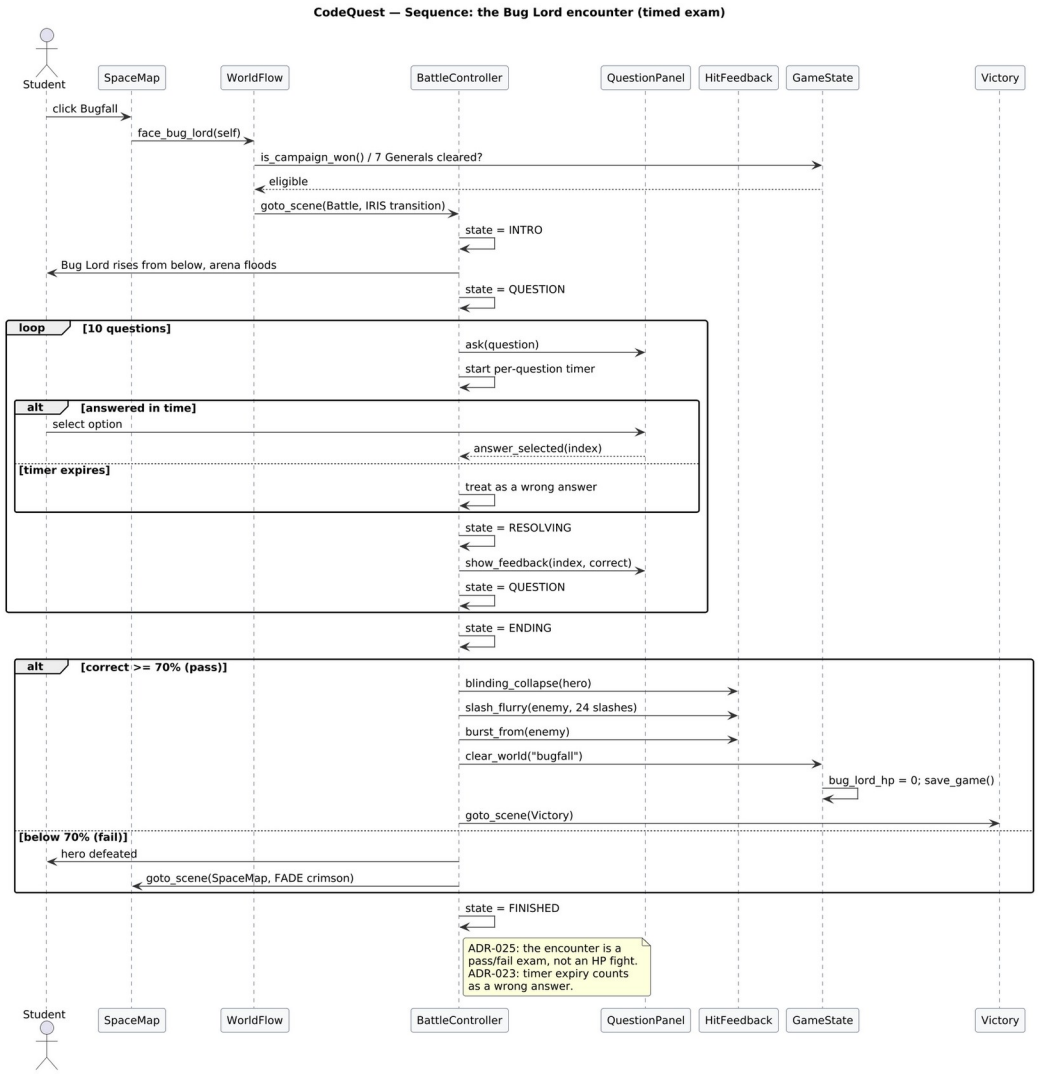


Figure 7: Sequence diagram: the Bug Lord timed examination

7.7 The battle state machine

`BattleController` is a five-state machine. The state is not decoration: it is the first of three guards that prevent a repeated or late input resolving twice.

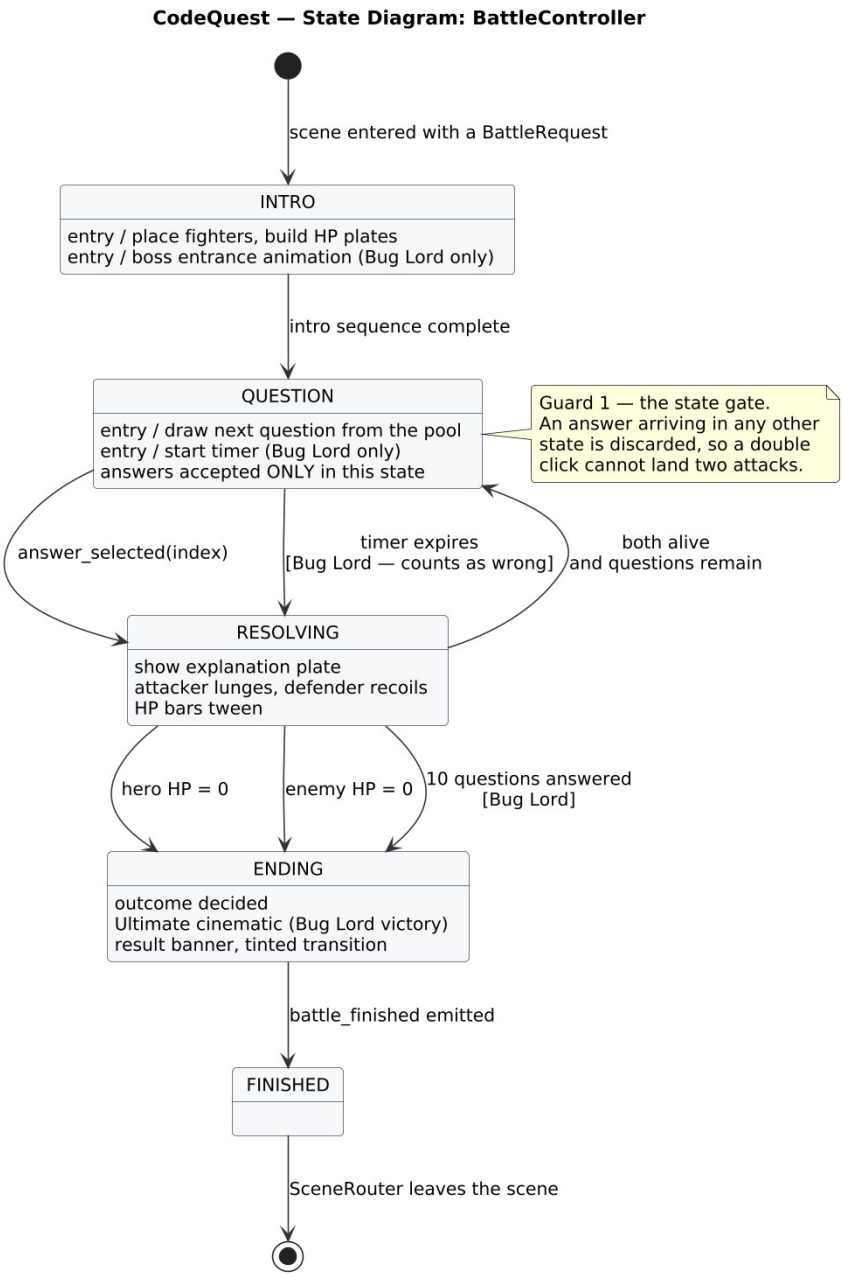


Figure 8: State diagram: BattleController

The three guards are worth stating explicitly, because the failure they prevent — two attacks landing from one question — is invisible until it happens:

1. **The state gate.** Answers are accepted only in `QUESTION`. An answer arriving in `RESOLVING` is discarded.
2. **Panel acceptance.** The question panel stops accepting input the moment an option is chosen, before the controller has even been notified.

3. **Awaited sequencing.** The resolution is a coroutine; the next question is not drawn until the animation, the explanation hold and the health bar tween have all completed.

7.8 Data flow

The context diagram shows the system’s boundary: a student, a content author, three content stores and two user-local stores.

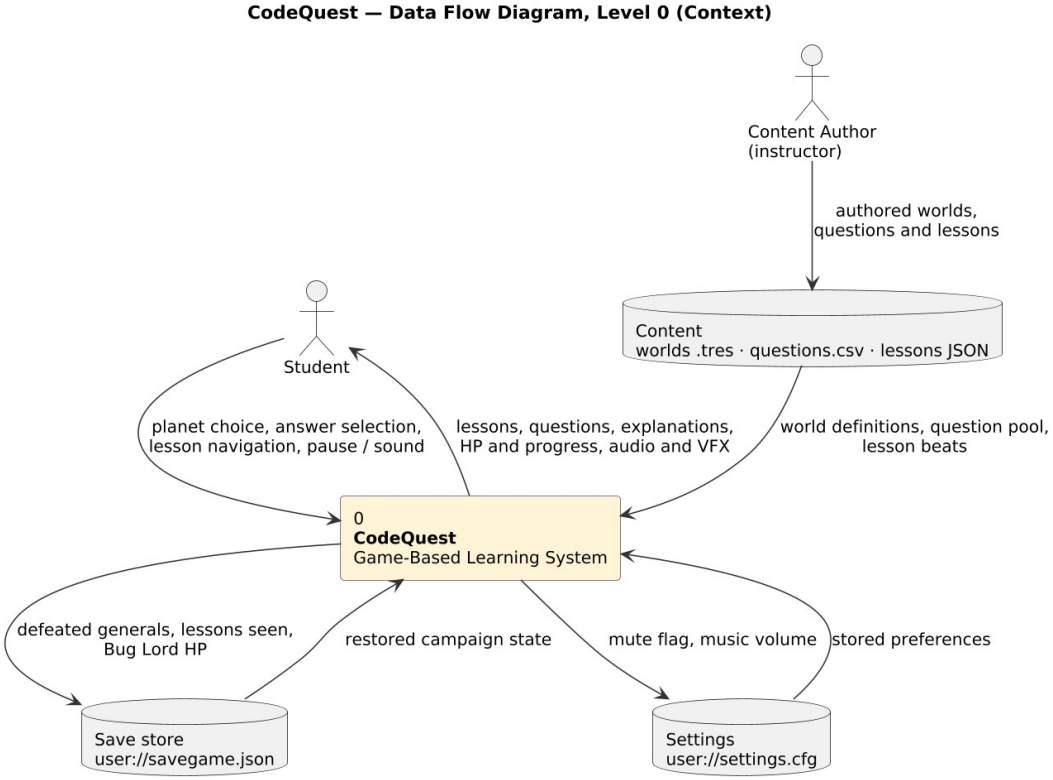


Figure 9: Data flow diagram, level 0 — context

Decomposing to level 1 gives six processes. Note that process 5.0, Update Progress, is the only process that writes to the campaign state store — a deliberate single point of persistence.

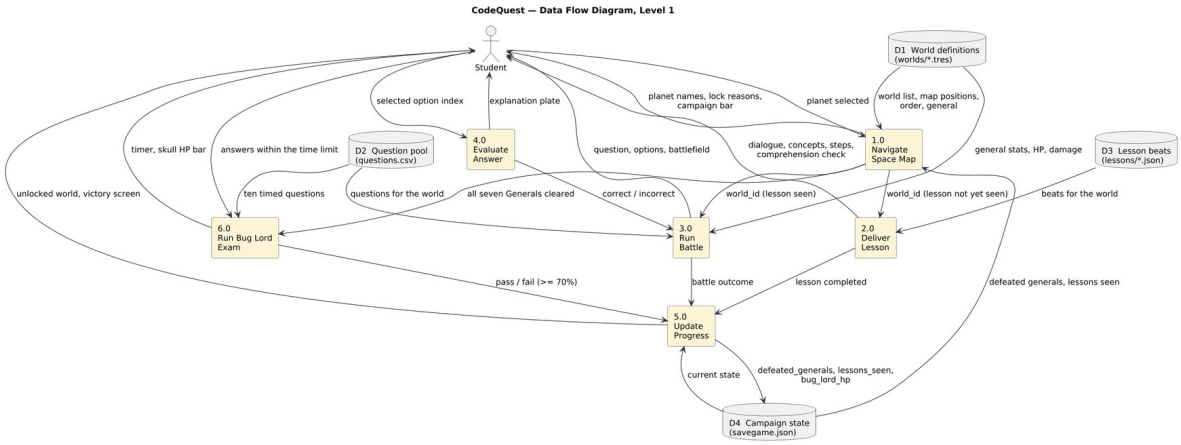


Figure 10: Data flow diagram, level 1 — six processes and four data stores

7.9 Data model

Content is modelled as Godot resources rather than rows in a database, but the entity relationships are conventional and are shown below. The save file is the only mutable store.

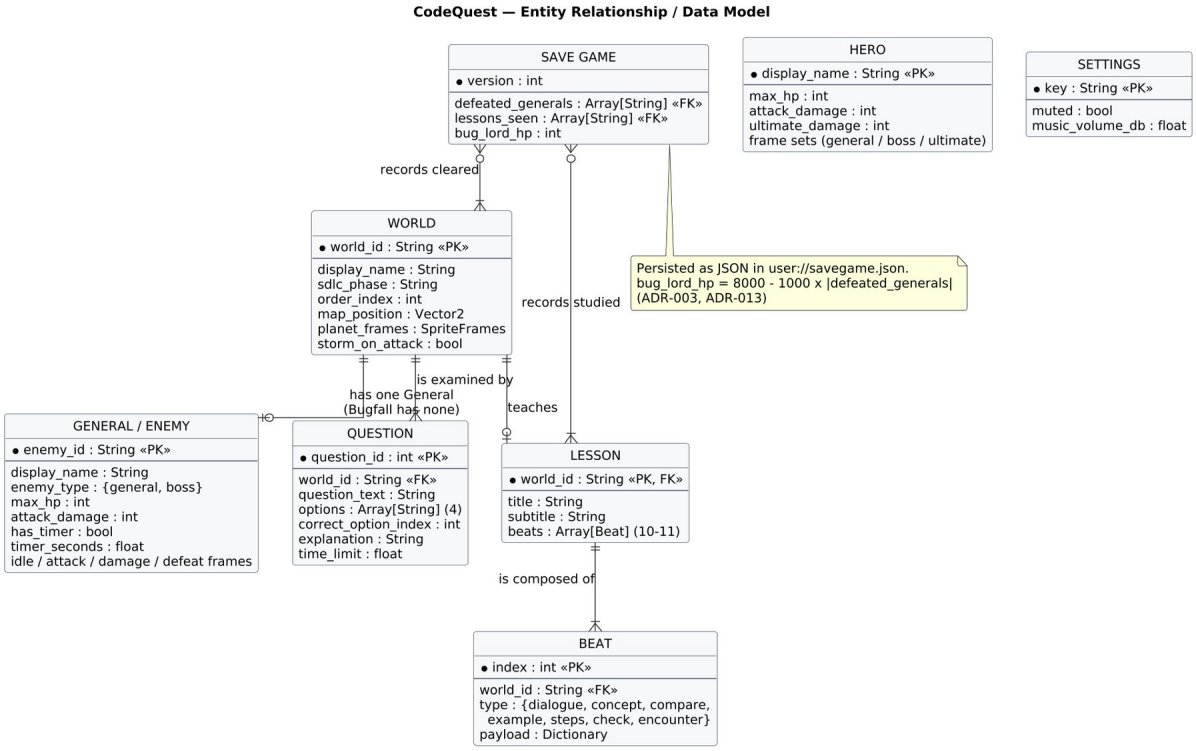


Figure 11: Entity relationship diagram

Two relationships carry design decisions. A world **may** have a General — the cardinality is zero-or-one, not one, because Bugfall has none. And the Bug Lord's health is not

stored independently of the cleared list; it is derived as 8000 minus 1000 times the number of worlds cleared, which is what makes clearing a world idempotent.

7.10 Principal algorithms and their invariants

Five decisions in the system are algorithmic rather than structural. Each is stated below with the invariant it preserves, because in every case the invariant is the reason the algorithm was chosen over the obvious alternative.

Algorithm	Method	Invariant preserved	Alternative rejected
World unlocking	A world is unlocked when its `order` index is no greater than `cleared.size()`. Evaluated on demand, O(1), no state stored.	The unlock set is always exactly consistent with the cleared set, because it is derived from it rather than stored beside it.	A per-world `unlocked` boolean, which is a second copy of the same fact and can drift out of step with the first.
Question selection	The world's pool is drawn into a `used` list; each draw samples uniformly from the complement, and the pool resets only when an encounter begins.	No question recurs within a single encounter; the draw is still random across playthroughs.	Sampling with replacement, which repeats questions inside one fight, and pre-shuffling the whole pool, which makes successive playthroughs identical.
Progression update	`clear_world(id)` appends to `cleared` only if the identifier is absent, then writes the save file.	Clearing a world is idempotent: replaying a cleared world cannot advance the campaign a second time.	An unguarded append or an increment counter, either of which inflates progress on a replay.
Bug Lord health	Derived at battle start as 8000 − 1000 × `cleared.size()`, never stored.	Boss health can never disagree with campaign progress, and the derivation inherits idempotency from the clause above.	Storing a mutable boss health field, which must then be kept in step with every progression write.
Animation timing	Every duration is computed as frame count divided by frame rate rather than awaited from a signal.	Timing is correct for all 41 `SpriteFrames` resources in the project, which are all set to loop.	`await animation_finished`, which Godot never emits for a looping animation, deadlocking the sequence.

Table 9: Principal algorithms, the invariant each preserves, and the alternative it was chosen over

The examination logic in Bugfall is a sixth case, and is a gate rather than an attrition model. Ten questions are asked under a per-question timer; an expired timer is recorded as a wrong answer, so the state machine has one failure path rather than two. A pass requires seven correct. The hero’s health is sized so that the fourth wrong answer empties it exactly, which makes the two failure conditions — failing the examination and losing the fight — coincide by construction rather than being checked separately.

8. Technology Stack

Layer	Technology	Version	Reason for the choice
Engine	Godot Engine	4.7.2 stable	Open source with no licence cost or revenue share; a scene-and-node model that matches the layered design; first-class 2D; exports to desktop from one codebase
Language	GDScript	Godot 4 syntax	Typed, integrated with the engine's signal and resource systems; static typing used throughout to catch errors at compile time
Renderer	OpenGL 3.3 Compatibility	—	Runs on integrated graphics; the Forward+ renderer offers nothing a 2D game needs
Content — worlds, enemies, hero	Godot `Resource` (`.tres`)	format 3	Editable in the inspector, diffable as text, loadable without code
Content — questions	CSV	—	129 rows of hand-authored questions; one file an instructor can edit in a spreadsheet
Content — lessons	JSON	—	Nested beat structures that CSV cannot express; hand-editable
Persistence	JSON in `user://`	schema v1	Human-readable for debugging; versioned so an incompatible save is refused
Settings	`ConfigFile` in `user://`	—	Engine-native key-value storage for mute state and music volume
Asset generation	Python 3 — NumPy, SciPy, Pillow, soundfile	—	The sixteen sound effects and five music loops are synthesised by script, not sourced, so they are original work and regenerable
Testing	Godot headless, custom runner	—	Five checks, 1159 assertions, no external framework
Version control	Git	—	Every change committed with a written message
Documentation	Markdown in-repo; PlantUML for diagrams	—	Versioned alongside the code; diagrams regenerate from text sources

Table 10: Technology stack and the justification for each choice

8.1 Why the content formats differ

Three content formats in one project invites the criticism of inconsistency, so the reasoning is worth stating. Worlds, enemies and the hero are **resources** because they are edited in the Godot inspector and referenced by other resources. Questions are **CSV** because there are 129 of them with identical flat structure, and a spreadsheet is the right tool for that shape. Lessons are **JSON** because a beat is a nested, polymorphic structure — a compare beat and a steps beat carry entirely different fields — which CSV cannot represent without becoming unreadable.

One engine behaviour had to be worked around. Godot's import system claims `.csv` files for translation and `.json` files for its own resource pipeline, with the result that the raw file is never packed into an exported build. Both formats are therefore accompanied by a sibling `.import` file specifying `importer="keep"`. This defect would have been invisible in the editor and fatal in the exported game; it is discussed further in Section 11.

9. Working Modules

The system comprises 26 GDScript files across four source packages and eight scenes. This section lists them; Section 10 describes each in detail.

Package	Module	Responsibility
scripts/data	world_data.gd	A world: identifier, display name, SDLC phase, General, planet frames, map position, order
	world_database.gd	The ordered collection of all eight worlds; lookup by identifier
	enemy_data.gd	A General or the Bug Lord: health, damage, animation frame sets, timer configuration
	hero_data.gd	The hero: health, attack and ultimate damage, two full frame sets plus the ultimate
	question_data.gd	One question: text, four options, correct index, explanation, time limit
	question_bank.gd	Parses the CSV and serves the pool for a given world
scripts/lesson	lesson_bank.gd	Loads and caches lesson JSON; reports whether a world has a lesson
	lesson_controller.gd	The beat player — seven renderers, typed text, navigation
	library.gd	The Bugfall Archive: a shelf of all eight lessons marking those studied
	world_flow.gd	The single place that decides what follows what
scripts/autoload	GameState.gd	Campaign progression and persistence
	SceneRouter.gd	Scene changes with transitions and payload passing
	AudioManager.gd	Sound effect pool, music crossfade, persisted settings, automatic button hookup
	PauseMenu.gd	Pause overlay available from any scene
scripts/battle	BattleController.gd	The encounter: state machine, question flow, damage, outcome
	battle_request.gd	The configuration object handed to a battle
	combatant.gd	A fighter: health, animation, anticipation, lunge, recoil, flash
	hit_feedback.gd	Damage numbers, screen shake, slashes, sparks, flashes, the Ultimate effects

Package	Module	Responsibility
	`nameplate.gd`	Name and health bar rendering for hero, General and Bug Lord
	`result_banner.gd`	Victory and defeat banner
	`lightning_strike.gd`	The storm effect used by worlds that declare it
`scenes`	`main.gd`	Title screen: continue, new game with confirmation, sound, quit
	`SpaceMap.gd`	The star map: planets, routes, hover information, lock reasons, campaign bar
	`RocketTravel.gd`	The travel sequence between worlds
	`QuestionPanel.gd`	Question presentation, option buttons, keyboard selection, explanation plate
	`Victory.gd`	The end-of-campaign screen listing all eight worlds and their phases

Table 11: Module inventory

10. Description of Each Module

10.1 The data layer

Every type in this layer extends Godot's `Resource` and carries `@export` fields only. The decision that content is data rather than code (ADR-001) is what permits a world to be added without touching gameplay logic.

10.1.1 WorldData and WorldDatabase

`WorldData` names a world and points at everything it needs: its General as an `EnemyData`, its planet animation frames, its position on the star map, and its order index. `WorldDatabase` holds the eight in campaign order. A world is unlocked when its order index is no greater than the number of worlds cleared, which makes unlocking a single arithmetic comparison rather than a stored flag that could drift out of step with progress.

10.1.2 QuestionBank and QuestionData

The bank parses one CSV into typed `QuestionData` objects and serves the pool for a requested world. Questions are drawn into a used list rather than sampled with replacement, which is what guarantees no repeat within an encounter.

10.1.3 LessonBank

Lesson JSON is loaded once and cached. Individual beats are deliberately left as dictionaries rather than promoted to typed objects: seven beat types with disjoint fields would require seven classes and a factory, and the only consumer is the renderer that already switches on type.

10.2 The session layer

10.2.1 GameState

Holds three pieces of state — the list of defeated generals, the list of lessons seen, and the Bug Lord's health — and persists them as versioned JSON. The most important method is `clear_world()`, which is idempotent: clearing an already-cleared world returns without changing anything. Without that guard a player could replay world one repeatedly and reduce the Bug Lord to zero without ever seeing worlds two to seven.

```
func clear_world(world_id: String) -> bool:
    if world_id in defeated_generals:
        return false                        # idempotent — no double damage
    defeated_generals.append(world_id)
    bug_lord_hp = max(0, bug_lord_hp - BUG_LORD_DAMAGE_PER_WORLD)
    save_game()
    progress_changed.emit()
    return true
```

Loading is strict rather than forgiving. A save whose `version` field does not match is refused outright and a fresh campaign started, rather than applying whichever fields happen to parse. A partially applied save is a worse failure than a lost one because it is silent.

10.2.2 SceneRouter

Offers two transition shapes — a fade and a circular iris — and an arbitrary tint. The choice is semantic: victory leaves a battle through gold, defeat through dark crimson, and the Bug Lord is entered through an iris, which is reserved for that one transition in the whole game so that it reads as significant.

10.2.3 AudioManager

Maintains a pool of ten audio stream players so that overlapping effects do not cut each other off, and two music players so that one track can fade out while the next fades in. Its most useful property is that it connects to the scene tree's `node_added` signal and hooks every `Button` in the game as it is created. No scene contains a single line of hover-sound wiring; a button wanting a different sound sets a metadata key.

Sixteen sound effects and five music loops were **synthesised in Python rather than sourced**, which makes them original work with no licensing question. They were also measured rather than judged by ear: spectral centroid and band-limited RMS between 250 Hz and 8 kHz were computed for every asset, which caught two defects a listening test had missed — effects voiced below 200 Hz that laptop speakers simply do not reproduce, and music mixed 13 dB too quiet in the band where it would actually be heard.

10.2.4 PauseMenu

A canvas layer above every scene but below the scene router's transition. It is suppressed on the title screen, the Space Map and the Archive, where pausing is meaningless. Implementing it required correcting all eleven `SceneTree.create_timer()` calls in the project: the engine default is `process_always = true`, which means a timer keeps running while the tree is paused — so before this change, pausing mid-battle would have paused the animation but not the countdown.

10.3 The battle layer

10.3.1 BattleController

The largest module in the project at roughly 28 KB, and the one that runs both encounter types. Its structure is the five-state machine of Section 7.7. Both the General fight and the Bug Lord examination are driven by the same loop; the mode flag on the `BattleRequest` selects the timer, the question count, the layout and the ending condition.

The Ultimate is the one place where presentation is sequenced explicitly rather than emerging from the loop. On the final blow against the Bug Lord the controller runs a seven-beat cinematic: a white flash that grows to fill the screen, a collapse of that light onto the hero, a flurry of twenty-four slash arcs across roughly two and a half seconds, the defeat animation, and a violet and white burst originating from the Bug Lord.

10.3.2 Combatant

Wraps one fighter’s sprite, health and animation. The attack is not a single tween but four phases, which is what gives the blow its weight: a wind-up backwards with a squash, a lunge forward easing *in* rather than out so that the fighter accelerates into the target, the attack animation, and a return. The defender recoils from the point of impact.

One engine behaviour shaped this module. All 41 `SpriteFrames` resources in the project have `loop = 1`, and Godot never emits `animation_finished` for a looping animation. Every animation timing in the game is therefore computed from frame count divided by frame rate rather than awaited.

10.3.3 HitFeedback

Owns everything that is not a character: rising damage numbers, screen shake, slash arcs, spark bursts, full-screen flashes and the Ultimate’s effects. The slash arc is drawn as a tapering polygon along a circular path; its geometry was rendered to a PNG and inspected before any of it was written in GDScript, which caught a first attempt that produced a fat teardrop rather than a blade.

10.3.4 QuestionPanel

Presents the question and four options, accepts mouse or number-key selection, and on answering reveals the explanation. Showing the explanation required hiding the options that were not chosen: measurement during development showed the explanation overflowing its scroll area by 53 pixels in **every** battle in the game, a defect that had never been reported because the overflowing text was simply clipped and nobody knew it was missing.

10.4 The presentation layer

10.4.1 SpaceMap

The campaign hub. Draws eight planets at authored positions, joins them with a dotted Catmull-Rom spline whose waypoints are offset by a seeded random walk so the route meanders without ever crossing itself, and renders the campaign progress bar. Hovering a planet raises an information panel naming the world, its General and, when locked, the world that unlocks it. Clicking a locked planet shakes it and plays a refusal sound — silence reads as a broken button.

10.4.2 LessonController

Renders the seven beat types, types text at roughly 18 milliseconds per character with a soft tick at 45 millisecond intervals, and supports both forward and backward navigation. The steps beat reveals one step at a time by fading alone, not sliding: a `VBoxContainer` recomputes its children’s positions on every layout pass and would fight a position tween.

10.4.3 Library

The Bugfall Archive. Each row is a horizontal container holding a marker label — a gold tick for studied, a dim dot for not — and a button carrying the title. The marker is a separate label rather than a prefix on the button text because the font is proportional, and padding a column with spaces in a proportional font leaves the titles visibly misaligned.

11. Test Plan and Test Suite

11.1 Strategy

Verification combines **five automated checks comprising 1159 assertions** with a **twenty-eight case manual suite**. The split is deliberate and is defended here rather than assumed: automation covers pure logic that can be evaluated without a rendering context, and everything visual is verified by eye against an expectation written before the test was run.

The project ran on a one-change-at-a-time protocol. Each change was followed by a written handoff naming the steps to perform, the observable result to expect, and the realistic failure modes with their symptoms; the change was not considered complete until the expected result was confirmed. This is manual testing, but it has two properties that ad-hoc clicking does not. The expectation was written **before** the test was run, so a test whose expectation is formed after seeing the output confirms nothing. And the blast radius of each change was one system, so when something broke the cause was in the diff that had just landed.

11.2 Risk analysis

The five automated checks were not chosen by coverage. They were chosen by multiplying the probability of a defect class by the cost of missing it.

Risk	Likelihood	Cost if missed	Mitigation
Editor-only correctness — works in the editor, fails when exported	High — two instances already found	Total: an exported game with no questions or no worlds is unplayable	An export build must be produced and played. Nothing else finds this class
Sequencing regression — a question appears before its animation finishes	Low — three independent guards	High: unplayable battle	Automated check on the state machine
Progression corruption — a world cleared twice, the Bug Lord over-damaged	Low — `clear_world()` is idempotent	High: the campaign can be finished from world one	Automated check T2
Content errors — a wrong answer key, a duplicated question	Medium — 129 hand-written rows	Medium: an educational game that teaches something false	Automated CSV validation T1
Save incompatibility — an older save loads partially	Low — the version field is checked	High: silent corruption	Mitigated by design; verified by T3

Risk	Likelihood	Cost if missed	Mitigation
Asset drift — a new sheet at a different scale	Medium	Low: visible immediately	Normalisation in code plus an art specification checklist

Table 12: Risk analysis driving the choice of automated checks

11.3 Automated test suite

The suite runs as a **scene**, not with the `--script` flag:

```
godot --headless res://tests/TestRunner.tscn
```

That distinction is not cosmetic, and the reason is recorded as defect 14 below. Autoloads are created only when a main scene runs, and `BattleController` references `GameState` at compile time — so under `--script` that script fails to compile, the check reads nothing, and the suite reports a pass having verified nothing at all.

Check	Asserts	Risk guarded
T1 Question bank integrity	All 129 rows parse; every row has four options; every correct index is 0–3; no question text appears twice; every enemy identifier referenced by a world has a non-empty pool	Content errors
T2 Progression arithmetic	Eight successive `clear_world()` calls take 8000 to 0; clearing an already-cleared world changes nothing; `is_world_unlocked` matches the cleared count	Progression corruption
T3 Save round-trip	Save then load reproduces state exactly; a corrupt file is refused rather than half-applied; a wrong version starts fresh	Save incompatibility
T4 Examination threshold	Seven correct of ten passes and six fails; the fourth wrong answer both fails the examination and empties a 100-health hero	The Bug Lord rules
T5 World database integrity	Eight worlds; order indices 0–7 with no gaps or duplicates; every world has a General; every planet has frames	The defect 7 class, permanently

Table 13: The five automated checks — 1159 assertions, 0 failures

T3 deliberately writes an invalid save and a future-version save, so a *passing* run prints `Parse JSON failed` and `save version 999.0 is not 1`. Those lines are the refusal logic working, not a fault — noted here because a reader of the output would reasonably assume otherwise.

11.4 Manual test suite

Twenty-eight cases that cannot be automated at reasonable cost. Each has a fixed expected result written in advance.

Group	Cases	Covers
Smoke	M1–M5	Launch, title screen, continue and new game, Space Map renders, quit
General fight	M6–M13	Lesson plays, all seven beat types render, battle starts, correct and incorrect answers resolve, explanation shows, victory clears the world
Bug Lord	M14–M18	Entry gated on seven Generals, timer counts and expires, threshold evaluated, Ultimate cinematic plays, victory screen reached
Bugfall	M19–M21	Archive lists eight lessons, marks studied, a lesson re-reads and returns
Persistence	M22–M24	Progress survives a restart, a corrupt save is refused, settings persist
Export	**M25–M28**	**The exported build launches, questions load, worlds load, a full world is completable. Not waivable**

Table 14: Manual test suite

11.5 Defect log

Fourteen defects were found during development. The table is included in full because the pattern it shows is the justification for the strategy in Section 11.1.

#	Defect	Root cause	Class
1	Layout squeezed into the upper-left 60%	Design resolution raised to 1920×1080 while all content was authored in 1152×648 coordinates	Configuration
2	`SceneRouter` autoload not found	An externally written script not yet in the identifier cache; a new script needs a project reload before it can be referenced	Tooling
3	Planets unclickable after a routing change	`SpaceMap` still called the replaced `SceneManager`	Incomplete migration
4	Planets blurry	Global linear texture filtering applied to deliberate pixel art	Configuration
5	Battle hung forever on the first attack	Every `SpriteFrames` has `loop = 1`; Godot never emits `animation_finished` for a	Engine behaviour

#	Defect	Root cause	Class
		looping animation	
6	Characters shrank when attacking	Sheets differ in frame height and in how much of the frame the character fills	Asset inconsistency
7	Space Map loaded a `SpriteFrames` as a `WorldData`	A directory scan matched every `.tres` in the worlds folder	Design
8	**Questions would have been absent from any exported build**	Godot's CSV translation importer claimed the file; the raw CSV is never packed	**Export-only**
9	Title art drew at full 1949×541	`size` set before `expand_mode`, so the minimum size clamped it straight back up	Engine behaviour
10	`WorldData.general` disappeared	A file rebuilt from a stale cached copy taken before the field was added	Process
11	Two shadowing warnings	A parameter named the same as a field	Hygiene
12	Bug Lord painted over the health bars and question panel	Combatants added to the tree after the UI; Godot draws in tree order	Engine behaviour
13	Starfield invisible	A child `ColorRect` painting over its own parent's `_draw`	Engine behaviour
14	**A test suite reported a pass having checked nothing**	T4 was run with `--script`, where autoloads do not exist, so `BattleController` failed to compile and no assertion executed	**Test infrastructure**

Table 15: Defect log

11.5.1 What the log says

- **Five of fourteen are the engine behaving correctly in a way the author did not expect** (2, 5, 9, 12, 13). No test suite would have predicted these; they were found by running the system and looking at it.
- **Two would only ever have appeared in an exported build** (7 and 8). Both were found by reading, not by running — the editor was perfectly happy. This is the most dangerous defect class in the project, and the risk analysis is built around it.
- **Defect 14 is the most instructive entry in the table.** A test that fails is useful. A test that silently skips itself and reports success is worse than no test at all, because it converts an unknown into a false assurance. T4 now records an explicit failure if it cannot load what it is meant to check.

- **Two were process failures rather than code failures** (1 and 10), both caused by acting on a stale copy of something.

11.6 Exit criteria

1. T1–T5 pass, or are consciously waived with the reason recorded.
2. M1–M24 pass in the editor.
3. **M25–M28 pass in an exported build. Not waivable.**
4. The engine error log is clear after a full playthrough.
5. The traceability matrix reflects the build being submitted.

11.7 What is deliberately not tested

- **Animation appearance and timing.** Verified by eye. Automating it would mean golden-image comparison, which is more machinery than a project of this size justifies.
- **Layout at non-default window sizes.** The `canvas_items` stretch mode makes this largely structural, and the game targets one aspect ratio.
- **Performance and memory.** Not measured. Video memory at the full character roster is an identified but untested risk.

12. Screenshots

All screenshots are from the running build. They are presented in the order a player encounters them.

12.1 Title and navigation

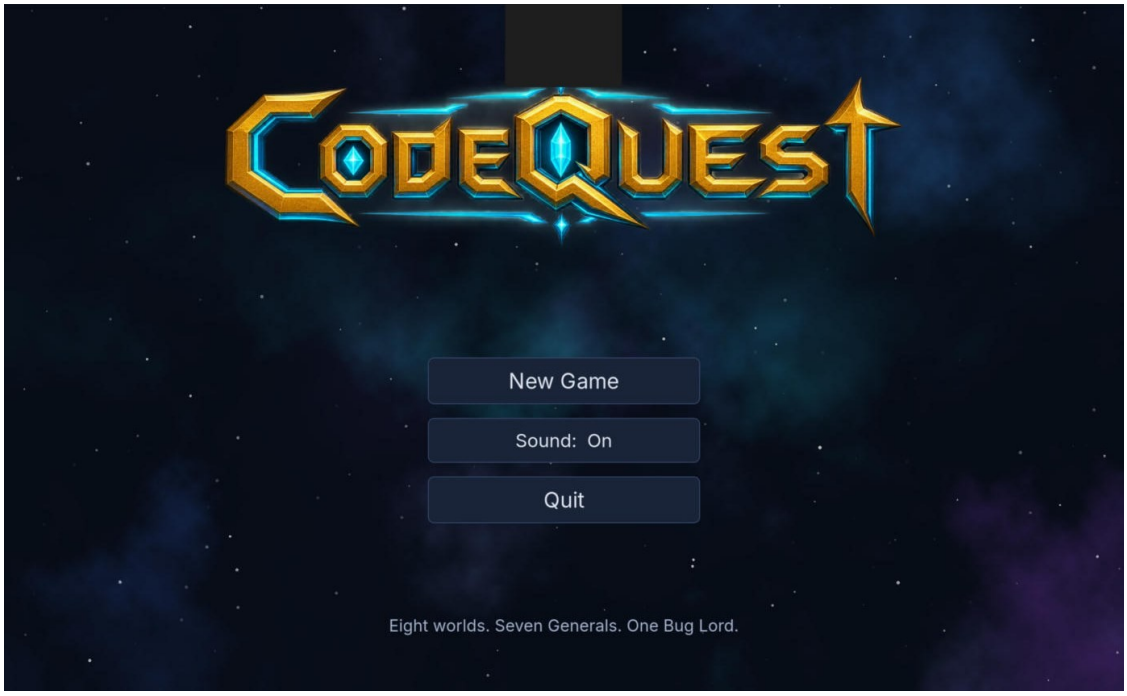


Figure 12: The title screen on a first launch. A Continue option appears once a campaign has been started.



Figure 13: The title screen mid-campaign, reporting worlds cleared and the Bug Lord's remaining health.

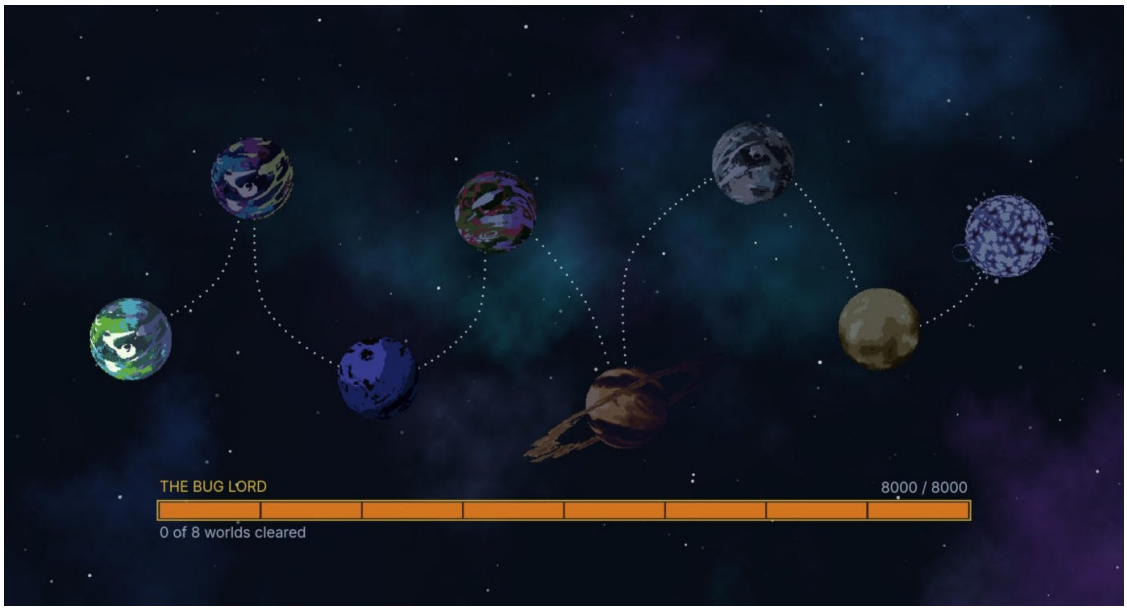


Figure 14: The Space Map at the start of a campaign. Locked worlds are dimmed but remain part of the composition; the campaign bar is full because the Bug Lord is undamaged.



Figure 15: Hovering a planet names the world and its General.



Figure 16: A locked world states which world unlocks it. The planet also shakes and a refusal sound plays — silence would read as a broken control.



Figure 17: The Space Map with seven worlds cleared. The campaign bar has fallen to 1000 of 8000 and turned red.

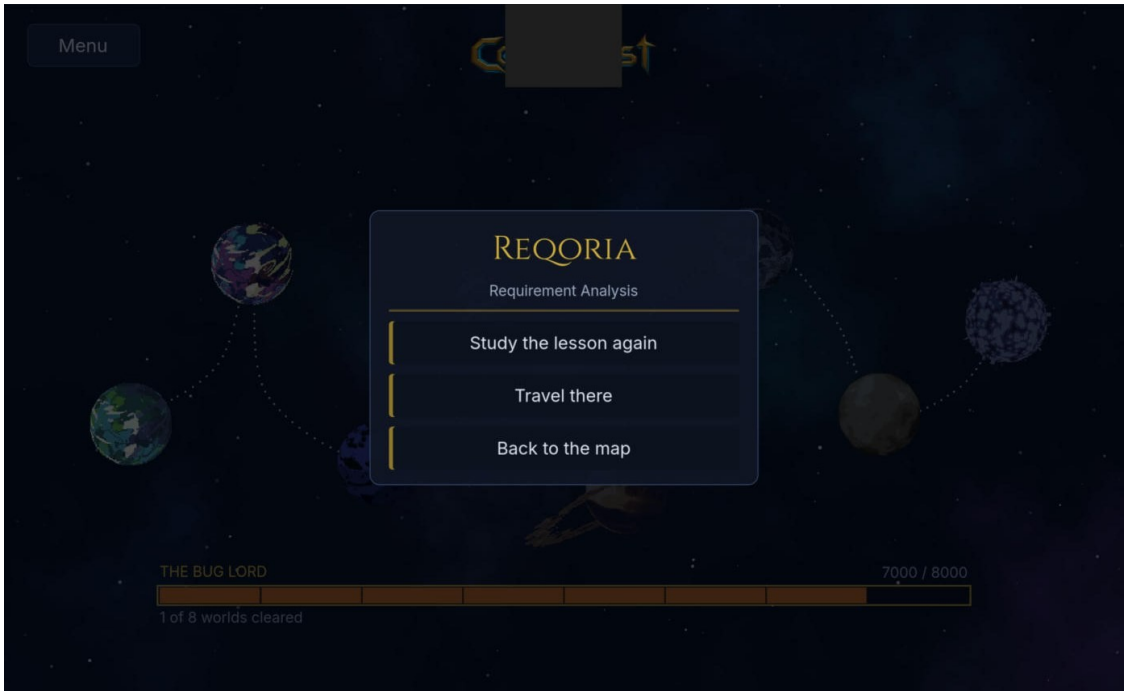


Figure 18: A world whose lesson has been studied offers the choice of re-reading it or travelling, so that material remains reachable without finishing the game.

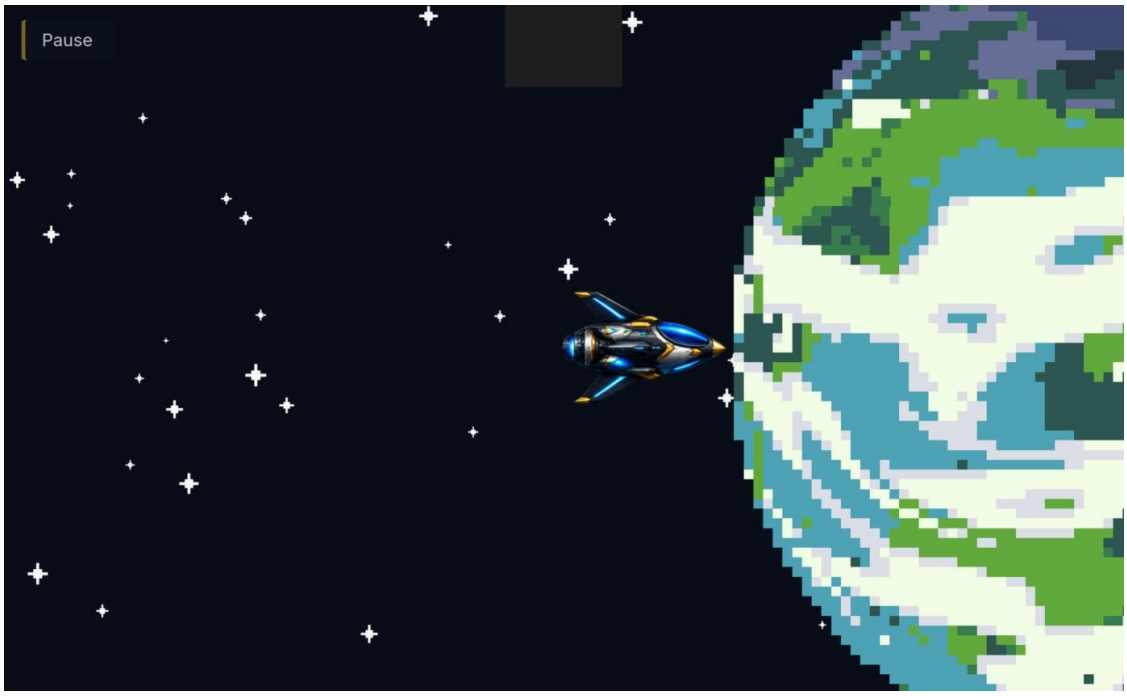


Figure 19: Rocket travel between worlds, arriving at Reqoria.

12.2 The teaching phase

Sifu, The Wise delivers a lesson of ten or eleven beats before each world’s first encounter. Five of the seven beat types are shown.



Figure 20: A dialogue beat. Sifu sets the stakes for the world in character; text is typed rather than appearing at once.



Figure 21: A concept beat: a titled definition with supporting prose and the key terms named beneath.



Figure 22: A compare beat, contrasting functional with non-functional requirements.



Figure 23: An example beat. The same request written ambiguously and then pinned down — the distinction the world is built around.



Figure 24: A steps beat. The five steps are revealed one at a time rather than all at once.



Figure 25: A check beat: a comprehension question inside the lesson, answered before any fight begins.

12.3 The battle

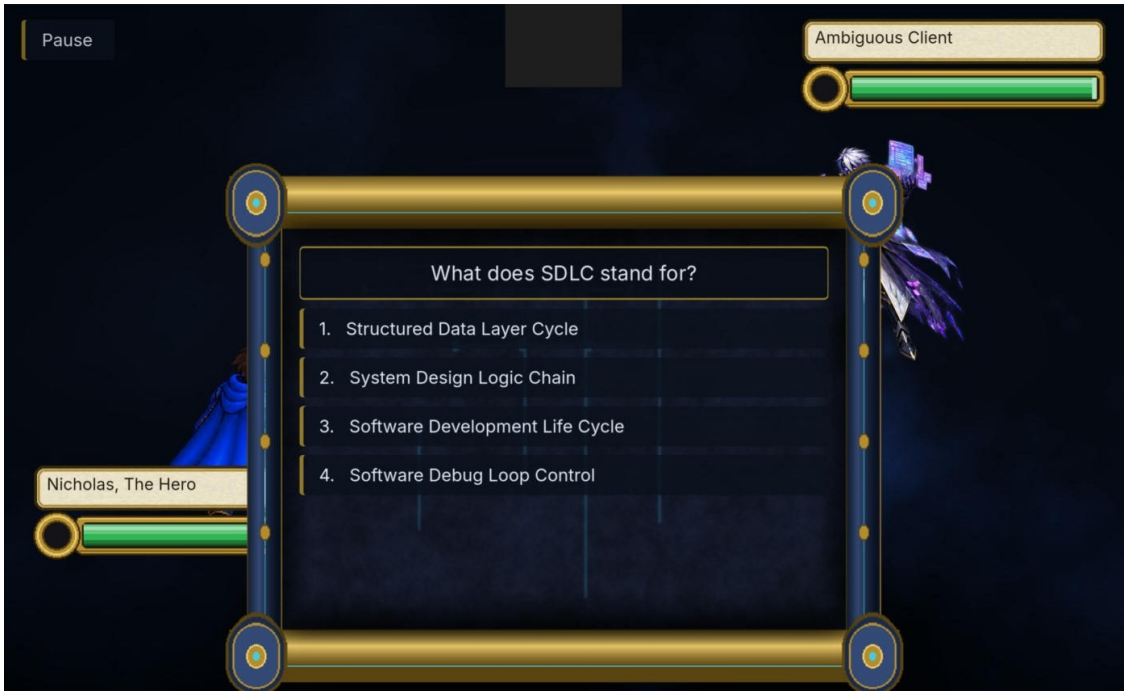


Figure 26: The question panel during a General fight. The battlefield dims behind it; options are selectable by mouse or by number key.

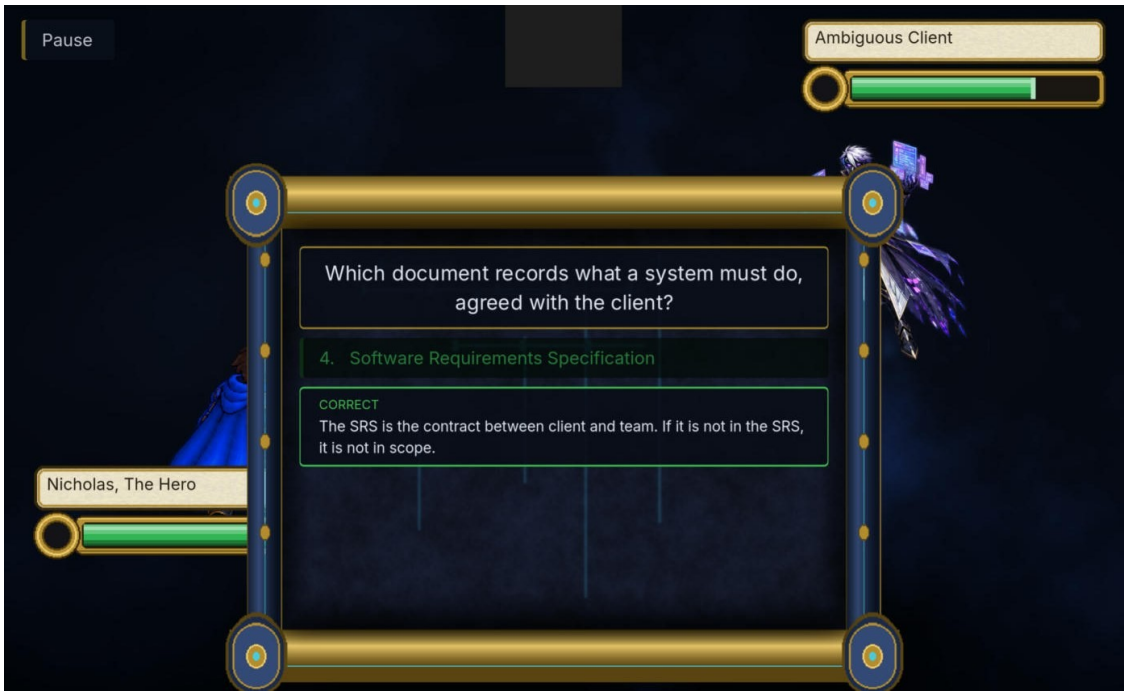


Figure 27: After answering, the chosen option is marked and the explanation appears. The options not chosen are hidden, which is what keeps the explanation inside its scroll area.

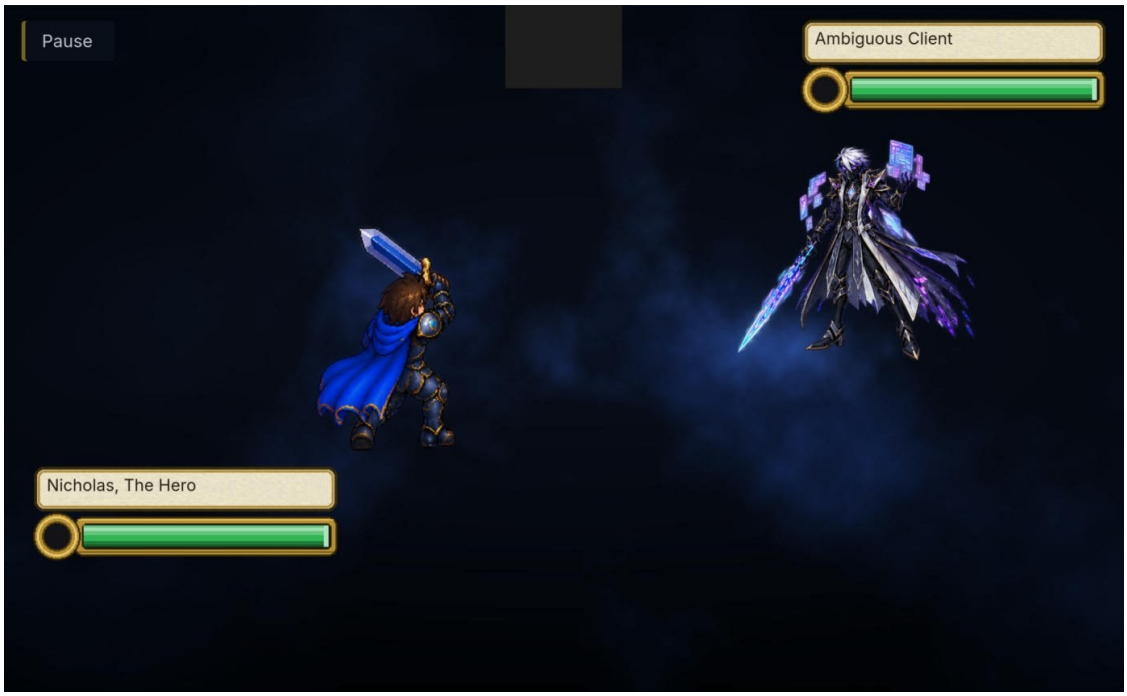


Figure 28: The battlefield between questions. The hero winds up to strike the Ambiguous Client of Reqoria; both health bars and the hero's nameplate are visible.



Figure 29: The pause menu, reachable by the Escape key or the on-screen control from any scene where pausing is meaningful.

12.4 Bugfall, the Bug Lord and victory



Figure 30: The Bugfall Learning Archive. All eight lessons are listed, with those already studied marked, and the header reports how many of the eight have been read.



Figure 31: The Bug Lord encounter. The skull-framed health bar spans the top of the screen and the per-question countdown sits in the upper right.



Figure 32: The Ultimate landing. The screen floods white, the light collapses onto the hero, and a flurry of slash arcs falls across roughly two and a half seconds — 1000 damage, the whole of the Bug Lord's remaining health, in a single blow.



Figure 33: The victory screen, listing all eight worlds and the SDLC phase each one taught — the syllabus the player has just completed, restated as a record of the campaign.

13. Conclusion

CodeQuest set out to restructure an eight-phase software engineering syllabus as a playable campaign in which assessment is the game mechanic rather than an interruption of it. That system exists and is complete end to end: eight worlds with distinct art, questions and lessons; a mentor-delivered teaching phase before every encounter; a turn-based battle driven entirely by multiple-choice assessment; a reference archive; a timed final examination; and progression that persists between sessions.

Four results are worth drawing out.

The architecture held the line it was designed to hold. The objective stated in Section 7.1 was that adding a world must not require changing gameplay code. It does not: a world is a resource file, a block of rows in the question bank and a JSON lesson. The three-layer model with one-way dependencies, the four justified autoloads and the single flow class are what make that true, and 44 architecture decision records document why each was chosen.

The verification strategy was shaped by evidence rather than by convention. Five automated checks were chosen by multiplying defect probability by cost, not by pursuing coverage. The defect log justifies that choice: five of fourteen defects were the engine behaving correctly in an unexpected way, which no unit test would have predicted, and two would have appeared only in an exported build. The most instructive entry is defect 14, a test that reported success having executed no assertions — a reminder that a test which silently skips itself is worse than no test, because it converts an unknown into a false assurance.

The design occupies a specific, documented position in the literature. Software process is the most studied area of game-based software engineering education (Souza et al., 2018) and SimSE has modelled it since 2004, so the contribution is not coverage of the process itself. It is the combination of four properties set out in Section 4.3: content gamification, in which the assessment instrument is the mechanic rather than a reward wrapped around one, which appears in no study reviewed by Tonhão et al.; a narrative role-playing structure, reported by the same review as rarely explored; explicit instruction delivered before every assessment, whose absence SimSE’s own evaluation identifies as a source of difficulty; and full worlds given to Maintenance and Configuration Management. Sections 5 to 10 are the implementation of those four conditions.

The technical contributions are reusable independently of the subject matter. Four are worth naming. The flow layer (Section 7.5) reduces every transition in the campaign to one pure function of the game state, so the sequencing rules live in a single file rather than being distributed across eight scenes. The content pipeline (Section 8.1) selects three storage formats by editing profile — resources for structure, CSV for tabular bulk, JSON for nested authoring — rather than forcing one format on all three. The progression guard (Section 16.3) makes world completion idempotent, so a replayed world cannot inflate

campaign state. And the battle controller (Section 10.3.1) drives two structurally different encounters — an attrition fight and a timed pass/fail examination — from one five-state machine parameterised by a request object. None of the four depends on the questions being about software engineering; the same engine would carry any multiple-choice syllabus with a phase structure.

14. Future Work

14.1 Evaluation

No controlled study was conducted within this project's scope, so the system's educational effect is unmeasured. A between-subjects study — one cohort playing CodeQuest, one receiving equivalent instruction conventionally, with a pre-test, an immediate post-test and a **delayed** post-test several weeks later — would turn a designed artefact into evidence. The delayed measure matters most: Sailer and Homner specifically note that delayed post-tests are rare in the gamification literature, which is precisely why its retention claims are weak. SimSE's evaluation also shows what to measure besides scores — completion rate and drop-off, since its central finding was that students who played learned, and students did not play.

14.2 Known defects and limitations

- **The game has not been exported and played end to end.** Two defects in the log would have appeared only in an exported build, so until M25–M28 are executed no requirement in the matrix is verified outside the editor.
- **Camera zoom during combat** is the one visual-polish requirement still open.
- **Video memory at the full character roster** is an identified but unmeasured risk.
- **Quitting to the title mid-battle** frees a scene whose coroutines are still awaiting timers; whether the engine logs an error in this case was never verified.

14.3 Content and features

- **A larger question bank.** 129 questions across eight worlds means a player who replays a world sees repeats. Three to four hundred would remove that entirely.
- **Difficulty settings.** The current damage values suit a student meeting the material for the first time; a harder mode for revision would widen the audience.
- **An instructor dashboard.** The save file already records which lessons were studied and which worlds cleared. Aggregating that across a class, and recording which questions are most often answered wrongly, would turn the game into a diagnostic instrument — the single feature most likely to make it useful to a department rather than to an individual.
- **A content-authoring interface.** Content is currently authored by editing CSV, JSON and resource files. That is a reasonable ask of a developer and an unreasonable one of a lecturer.
- **Accessibility.** Colour-blind-safe palettes, adjustable text size, and a setting to disable screen shake and flashes. The Ultimate cinematic in particular is a sustained high-contrast flash sequence.

- **Web export.** Godot exports to WebAssembly; a browser build would remove installation as a barrier to classroom use entirely.

14.4 Research directions

Two questions arise directly from the design. First, does content gamification outperform structural gamification on the same material? CodeQuest and a points-and-badges quiz over an identical question bank would isolate the variable that Tonhão et al. identify as unstudied. Second, does narrative framing affect retention of process knowledge specifically? The same campaign stripped of its story and characters, with the questions and lessons unchanged, would answer it.

15. References

References are listed alphabetically by first author. Items marked with a dagger (†) were verified against the publisher record but one or more bibliographic fields could not be retrieved at the time of writing; these are flagged rather than silently completed.

Almeida, C., Kalinowski, M., Uchôa, A. and Feijó, B. (2023) ‘Negative Effects of Gamification in Education Software: Systematic Mapping and Practitioner Perceptions’, *Information and Software Technology*. doi:10.1016/j.infsof.2022.107142.

Gui, Y., Cai, Z., Yang, Y., Kong, L., Fan, X. and Tai, R. H. (2023) ‘Effectiveness of digital educational game and game design in STEM learning: a meta-analytic review’, *International Journal of STEM Education*, 10, article 36. doi:10.1186/s40594-023-00424-9.

Ishaq, K., Mat Zin, N. A., Rosdi, F., Jehanghir, M., Ishaq, S. and Abid, A. (2021) ‘Mobile-assisted and gamification-based language learning: a systematic literature review’, *PeerJ Computer Science*, 7, e496. doi:10.7717/peerj-cs.496.

Kosa, M., Yilmaz, M., O’Connor, R. V. and Clarke, P. M. (2016) ‘Software Engineering Education and Games: A Systematic Literature Review’, *Journal of Universal Computer Science*, 22(12), pp. 1558–1574. doi:10.3217/jucs-022-12-1558.

Kroustalli, C. and Xinogalos, S. (2021) ‘Studying the effects of teaching programming to lower secondary school students with a serious game: a case study with Python and CodeCombat’, *Education and Information Technologies*, 26(5), pp. 6069–6095. doi:10.1007/s10639-021-10596-y.

Navarro, E. O. and van der Hoek, A. (2004) ‘SimSE: An Interactive Simulation Game for Software Engineering Education’, in *Proceedings of the 7th IASTED International Conference on Computers and Advanced Technology in Education (CATE 2004)*.

Navarro, E. O. and van der Hoek, A. (2007) ‘Comprehensive Evaluation of an Educational Software Engineering Simulation Environment’, in *Proceedings of the 20th Conference on Software Engineering Education and Training (CSEET 2007)*. IEEE.

Sailer, M. and Homner, L. (2020) ‘The Gamification of Learning: a Meta-analysis’, *Educational Psychology Review*, 32(1), pp. 77–112. doi:10.1007/s10648-019-09498-w.

Souza, M. R. de A., Veado, L., Moreira, R. T., Figueiredo, E. and Costa, H. (2018) ‘A systematic mapping study on game-related methods for software engineering education’, *Information and Software Technology*. †

Tonhão, S. de F., Herculani, J., Shigenaga, M. Y., Medeiros, A. S. S., Amaral, A. M. M. M., Silva, W., Colanzi, T. E. and Steinmacher, I. F. (2023) ‘Gamification in Software Engineering Education: a Tertiary Study’, in *Proceedings of the XXXVII Brazilian Symposium on Software Engineering (SBES 2023)*, pp. 358–367.

Wang, T., Trimble, M. and Brown, C. (2025) ‘DevCoach: Supporting Students Learning the Software Development Life Cycle with a Generative AI Powered Multi-Agent System’, in *Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion ’25)*. ACM.

15.1 Software and tools

Godot Engine (2026) *Godot Engine 4.7.2 stable*. Available at: https://godotengine.org

CodeCombat. Available at: https://www.codecombat.com

CodinGame. Available at: https://www.codingame.com

Tomorrow Corporation (2015) *Human Resource Machine*.

Duolingo. *How to learn a language on Duolingo*. Available at: https://blog.duolingo.com

PlantUML. Available at: https://plantuml.com

15.2 Project documents

The following are maintained in the project repository and are the primary sources for Sections 6 to 11 of this report.

- `docs/ARCHITECTURE.md` — architecture overview, layer model, principal flows
- `docs/DECISIONS.md` — 44 architecture decision records
- `docs/REQUIREMENTS_TRACE.md` — the 137-item traceability matrix
- `docs/TEST_STRATEGY.md` — risk analysis, automated and manual suites, defect log
- `docs/ART_SPEC.md` — the visual contract for all generated assets

16. Appendix: Implementation

Six listings are reproduced here, chosen because each demonstrates a design decision discussed earlier rather than because it is long. The complete source is in the project repository.

16.1 Project structure

```
new-game-project/
  project.godot                 # four autoloads registered here
  scripts/
    autoload/   GameState.gd  SceneRouter.gd  AudioManager.gd  PauseMenu.gd
    data/       world_data.gd  world_database.gd  enemy_data.gd
                hero_data.gd  question_data.gd  question_bank.gd
    lesson/     lesson_bank.gd  lesson_controller.gd  library.gd  world_flow.gd
    battle/     BattleController.gd  battle_request.gd  combatant.gd
                hit_feedback.gd  nameplate.gd  result_banner.gd
                lightning_strike.gd
  scenes/       main/  space_map/  travel/  lesson/  battle/  victory/
  resources/    worlds/*.tres  enemies/*.tres  hero/*.tres
  data/         questions.csv  lessons/*.json   (+ .import files)
  tests/        TestRunner.tscn  run_tests.gd
  tools/        make_sfx.py  make_music.py  validate_lessons.py
  docs/         ARCHITECTURE.md  DECISIONS.md  REQUIREMENTS_TRACE.md
                TEST_STRATEGY.md  ART_SPEC.md  diagrams/
```

16.2 The flow layer — world_flow.gd

The single place that decides what follows what. Every caller wanting to enter a world calls `arrive()`; the rule that a lesson precedes a battle lives here and nowhere else.

```
class_name WorldFlow

const LESSON_SCENE  := "res://scenes/lesson/Lesson.tscn"
const LIBRARY_SCENE := "res://scenes/lesson/Library.tscn"
const BATTLE_SCENE  := "res://scenes/battle/Battle.tscn"
const LIBRARY_WORLD := "bugfall"

## Entering a world: lesson first if there is one and it has not been seen.
static func arrive(from: Node, world: WorldData) -> void:
    if LessonBank.has_lesson(world.world_id) \
            and not GameState.has_seen_lesson(world.world_id):
        _goto(from, LESSON_SCENE, func(s): s.setup(world))
        return
    after_lesson(from, world)

## What follows a lesson: the Archive for Bugfall, otherwise the fight.
static func after_lesson(from: Node, world: WorldData) -> void:
    if world.world_id == LIBRARY_WORLD:
        _goto(from, LIBRARY_SCENE)
        return
    _goto(from, BATTLE_SCENE, func(s): s.setup(BattleRequest.for_world(world)))
```

16.3 Progression — the idempotency guard

Discussed in Section 10.2.1. Without the first two lines a player could replay world one until the Bug Lord was dead.

```
const BUG_LORD_MAX_HP          := 8000
const BUG_LORD_DAMAGE_PER_WORLD := 1000

func clear_world(world_id: String) -> bool:
    if world_id in defeated_generals:
        return false                        # idempotent
    defeated_generals.append(world_id)
    bug_lord_hp = maxi(0, bug_lord_hp - BUG_LORD_DAMAGE_PER_WORLD)
    save_game()
    progress_changed.emit()
    return true

var current_world_index: int:
    get: return defeated_generals.size()

func is_world_unlocked(order_index: int) -> bool:
    return order_index <= current_world_index
```

16.4 The battle state gate

The first of the three guards described in Section 7.7. An answer arriving in any state other than `QUESTION` is discarded rather than queued.

```
enum State   { INTRO, QUESTION, RESOLVING, ENDING, FINISHED }
enum Outcome { VICTORY, HERO_DEFEATED }

signal battle_finished(outcome: Outcome, request: BattleRequest)

func _on_answer_selected(index: int) -> void:
    if state != State.QUESTION:            # guard 1 — the state gate
        return
    state = State.RESOLVING

    var correct := index == _current.correct_option_index
    _panel.show_feedback(index, correct, feedback_hold)
    await _exchange_blow(correct)

    if state == State.ENDING or state == State.FINISHED:
        return
    state = State.QUESTION
    _ask_next()
```

16.5 Automatic audio wiring

Described in Section 10.2.3. Connecting to the tree's `node_added` signal means no scene in the game contains hover-sound wiring; a button wanting a different sound sets a metadata key.

```
const META_HOVER := "hover_sfx"
const META_CLICK := "click_sfx"

func _ready() -> void:
    get_tree().node_added.connect(_on_node_added)

func _on_node_added(node: Node) -> void:
    if node is Button:
        hook_button(node)

func _button_hover(b: Button) -> void:
    if b.disabled:                  # a disabled Button still emits
mouse_entered
        return
    var sound: StringName = b.get_meta(META_HOVER, &"ui_hover")
    play(sound, 0.07, -7.0)
```

16.6 A lesson beat

One beat from `data/lessons/reqoria.json`, showing the compare type rendered in Figure 14.

```
{
  "type": "compare",
  "title": "Two kinds, and the difference matters",
  "left_heading":  "Functional",
  "left_lead":     "What the system does.",
  "left_items": [
    "The user can reset a password",
    "The admin can revoke access",
    "A receipt is sent after payment"
  ],
  "right_heading": "Non-functional",
  "right_lead":    "How well it does it.",
  "right_items": [
    "Responds within 200ms",
    "Survives 1000 users at once",
    "Available 99.9% of the month"
  ]
}
```

16.7 Running the automated suite

```
# As a SCENE, not with --script — see defect 14, Section 11.5
godot --headless res://tests/TestRunner.tscn

# A passing run reports each check in turn and ends with the total:
#   T1 question bank integrity .......... OK
#   T2 progression arithmetic ........... OK
#   T3 save round-trip .................. OK
#   T4 examination threshold ............ OK
#   T5 world database integrity ......... OK
#   ----------------------------------------
```

```
#   1159 assertions, 0 failures
#
# T3 deliberately writes an invalid save and a future-version save, so a
# PASSING run also prints "Parse JSON failed" and "save version 999.0
# is not 1". Those lines are the refusal logic working.
```